Sound Omission of Type Guards and Type Arguments in Hammer Translations of Dependent Type Theory to Untyped First-Order Logic

Abstract

Hammer-style automation for proof assistants based on the Calculus of Inductive Constructions (CIC) translates goals and premises into untyped first-order logic and discharges them with automated theorem provers. The standard translation relativizes every quantifier with a *type guard* $\mathsf{T}(x, \tau)$ and applies every polymorphic constant to explicit *type arguments*; both are a major source of clause bloat and are known to degrade automated-prover performance. We formulate precise, syntactic criteria for omitting (i) type guards whose bodies are *covered* by a hypothesis atom that forces the type of the guarded variable (so that, e.g., $\forall x.\ \mathsf{T}(x, nat) \to Even(x) \to \phi$ may be replaced by $\forall x.\ Even(x) \to \phi$), and (ii) type arguments of function and constructor symbols that are recoverable from the type of a retained term argument (so that $cons(\alpha, x, l)$ may be replaced by $cons(x, l)$ while $nil(\alpha)$ must keep its argument). We prove both criteria sound, and the guard criterion moreover conservative, with respect to a polymorphic first-order fragment of CIC with inductive types and inductive predicates. Soundness is established model-theoretically: from any many-sorted model of the source theory we construct an untyped model with explicit ill-typed ("junk") elements that validates the translation together with a set of *typehood-completion axioms*; guard omission is then a logical equivalence relative to the completion axioms, and type-argument erasure preserves truth in the constructed model. We further show the criteria are tight: relaxing any of their clauses admits translations that derive goals false in the source theory. The criteria are directly implementable as a post-processing pass over the clause forms produced by existing hammer translations.

Contents

1 Introduction

Hammers [BKPU16] connect proof assistants to automated theorem provers (ATPs): given a goal, they select a set of premises from the ambient library, translate goal and premises into the ATP's logic — usually untyped first-order logic (FOL) with equality — run external provers, and reconstruct a found proof inside the proof assistant. For proof assistants based on the Calculus of Inductive Constructions (CIC), such as Rocq (formerly Coq), the translation step must bridge a large gap: CIC is a dependently typed higher-order logic, while the target is untyped and first-order. The CoqHammer translation [CK18, Cza18] bridges it by, among other devices, two systematic sources of formula growth:

(a) *Type guards.* A quantification $\forall x{:}\tau.\ \phi$ of the source logic is translated to $\forall x.\ \mathsf{T}(x, [\![\tau]\!]) \to [\![\phi]\!]$, where T is a binary *typing predicate* of the target logic; dually $\exists x{:}\tau.\ \phi$ becomes $\exists x.\ \mathsf{T}(x, [\![\tau]\!]) \wedge [\![\phi]\!]$. Guards keep the untyped provers from instantiating quantifiers with terms of the wrong type, which is in general necessary for soundness, but they lengthen every clause, enlarge the search space, and are a well-documented performance liability [MP08, BBPS16, CLS11].

(b) *Type arguments.* A polymorphic constant or constructor is translated together with its type instantiations: the list constructor becomes a ternary symbol in $\mathit{cons}(\alpha, x, l)$. Type arguments inflate every term occurrence and interact badly with term indexing in ATPs [MP08, BBPS16].

Both kinds of annotation are often *redundant*. If a premise has the form $\forall x{:}\mathit{nat}.\ \mathit{Even}\, x \to \phi$, then the guard $\mathsf{T}(x, \mathit{nat})$ is superfluous *given* the hypothesis atom $\mathit{Even}(x)$: in the intended interpretation the atom $\mathit{Even}(x)$ can only hold when x denotes a natural number, because *Even* is a predicate *on* natural numbers — an ill-typed application $\mathit{Even}\, t$ is not a proposition of the source logic at all, hence in particular not a provable one. Similarly, the type argument of *cons* is determined by the type of either of its term arguments, whereas the type argument of *nil* is determined by nothing that survives translation and must be kept. Proof-assistant practice supports the intuition — elaboration infers exactly these annotations — but turning the intuition into a criterion that is *provably* sound is delicate: the literature on type encodings for polymorphic first-order and higher-order logics contains both a series of soundness proofs for guard-based encodings [CLS11, BBPS16, BP11, CL07] and a matching series of concrete unsoundness examples for naive omissions [MP08, BBPS16]. For CIC the situation is harder still: types are themselves first-class terms, typing is not decidable by sort inference, and the only published soundness proof for a hammer translation of (a PTS approximation of) CIC [Cza18] leaves guard omission as a conjecture and poses the adaptation of monotonicity-based methods as an open problem.

Contributions. This paper gives precise criteria for both omissions and proves them correct for a substantial fragment of CIC. Concretely:

- We define a source calculus PFI — a polymorphic first-order fragment of CIC with parameterized inductive data types, inductive predicates, ML-style prenex type quantification, and equality — together with its *ground companion theory* $\mathbb{T}(E)$, a many-sorted first-order theory that captures exactly the reasoning about PFI-formulas that CIC licenses

(constructor injectivity, distinctness, case exhaustiveness, predicate introduction and inversion), and we make the connection to CIC precise (Section 4).

- We define the guarded translation $[\![\cdot]\!]$ of PFI into untyped FOL with equality, including the axiom classes emitted by hammer translations (typing axioms, unguarded injectivity and discrimination axioms, inversion axioms, translated premises) and a class of *typehood-completion axioms* $\mathrm{Comp}(E)$ asserting that provable atoms carry well-typed arguments. From an arbitrary many-sorted model $\mathfrak{N}$ of $\mathbb{T}(E)$ we construct an untyped model $\mathfrak{M}(\mathfrak{N})$ whose domain contains, besides tagged copies of the carriers of $\mathfrak{N}$, a syntactic universe of *junk* elements denoting ill-typed applications, and we prove that $\mathfrak{M}(\mathfrak{N})$ validates the whole translation, completion axioms and unguarded injectivity included (Section 5). This yields a self-contained, model-theoretic soundness theorem for the guarded translation: if the translated problem is refutable, the goal holds in every model of $\mathbb{T}(E)$ (Theorem 5.9).

- We formulate the guard-omission criterion (Section 6). Its ingredients are the *typing consequences* $\mathrm{tc}(A)$ of an atom A — guard atoms forced by A's predicate symbol at each argument position — and a pair of mutually recursive syntactic judgments $\mathrm{jt}_{x,u}/\mathrm{jf}_{x,u}$ ("junk-true"/"junk-false") identifying formulas whose truth value is determined under every valuation that falsifies the guard $\mathsf{T}(x,u)$. The main theorem (Theorem 6.6) states that if $\mathrm{jt}_{x,u}(\phi)$ then

$$\mathrm{Comp}(E) \models (\forall x.\ \mathsf{T}(x,u) \to \phi) \leftrightarrow (\forall x.\ \phi),$$

  a logical equivalence relative to the completion axioms; guard omission is therefore sound at *any* polarity, in premises and goal alike, and, when the completion axioms are emitted, it is also *conservative*: no ATP proof is lost (Corollary 6.9). We also analyze the existential and polarity-restricted cases, where only an implication holds, and prove a simultaneous-omission theorem justifying the fixpoint algorithm used in practice (Proposition 6.12).

- We formulate the type-argument-erasure criterion (Section 7): a type argument position j of a function or constructor symbol is erasable iff the corresponding type variable occurs in the declared type of a retained term argument. We prove a uniqueness lemma in the style of Blanchette et al.'s "inferable type argument" lemma [BBPS16] — in our fragment, by an elementary injectivity property of type substitution — and construct from $\mathfrak{N}$ an untyped model $\mathfrak{M}_\varepsilon(\mathfrak{N})$ of the *erased* translation, in which the omitted arguments are recovered uniquely from the retained ones. Truth of the translated goal in $\mathfrak{M}_\varepsilon(\mathfrak{N})$ again

coincides with truth in $\mathfrak{N}$ (Lemma 7.7), which yields soundness of erasure (Theorem 7.9) and of the combination of both optimizations (Theorem 7.10); the combination requires that predicate symbols are never erased, and we show why.

- We prove the criteria *tight* (Section 8): dropping an uncovered guard, dropping an existential guard conjunct licensed only by junk-truth, erasing a result-only type argument, and erasing a predicate's type argument each admit a concrete, consistent environment in which the transformed translation refutes a goal that fails in some model of $\mathbb{T}(E)$. These four counterexamples are the formal counterparts of unsoundness patterns observed in practice [MP08, BBPS16, CK18].

The criteria are designed to be implementable as a purely syntactic post-processing pass over the first-order formulas produced by an existing hammer translation, requiring from the source system only the declared types of the translated constants. We discuss the relation to guard-based and monotonicity-based type encodings, to Mizar's cluster registrations, and to the proof-theoretic soundness framework of [Cza18] in Section 9, together with the limitations of the fragment (higher-order features, dependent function types, conversion) and the corresponding open problems.

2 Overview

Before the formal development we walk through the four running examples, written in Rocq-like syntax on the left and in the guarded translation on the right. Throughout, $\mathsf{T}(t, u)$ is the typing predicate of the target logic ("t has type u"), and predicates and functions are translated to first-order symbols applied to their type and term arguments.

A covered guard. The premise $\forall x{:}nat.\ Even\, x \to Even\, (S\, (S\, x))$ translates to

$$\forall x.\ \mathsf{T}(x, nat) \to Even(x) \to Even(S(S(x))). \tag{1}$$

The guard is redundant. In the intended interpretation, an atom $Even(t)$ is true only if the source application $Even\ t$ is a well-formed *and* provable proposition; since $Even : nat \to \mathsf{Prop}$, well-formedness forces $t : nat$. We say the atom $Even(x)$ *covers* the guard $\mathsf{T}(x, nat)$: under any valuation sending x outside nat, the atom is false and the body of (1) is vacuously true. Hence (1) and $\forall x.\ Even(x) \to Even(S(S(x)))$ have the same truth value in the intended model — an *equivalence*, not merely an implication, so the rewrite is legitimate at any polarity. Moreover the equivalence is provable in pure first-order logic from the *typehood-completion axiom*

$$\forall x.\ Even(x) \to \mathsf{T}(x, nat),$$

which is itself true in the intended model. This observation drives the architecture of Section 6: guard omission is a logical equivalence relative to a stock of completion axioms, and the semantic content of the soundness proof is concentrated in the single fact that completion axioms hold in the intended model.

An inversion axiom. For the inductive predicate *le* the translation emits an inversion axiom

$$\forall n\, m.\, \mathsf{T}(n, nat) \to \mathsf{T}(m, nat) \to le(n, m) \to \big(m = n \vee \exists k.\, \mathsf{T}(k, nat) \wedge le(n, k) \wedge m = S(k)\big).$$

Both universal guards are covered by the hypothesis atom $le(n, m)$ and may be dropped. Note that the criterion must look *past* the covering atom, not into the disjunction: the disjunct $m = n$ contains m "naked" in an equation, and equations force no typing (junk equals junk). The recursive judgment jt of Section 6 is therefore satisfied at the implication whose antecedent is $le(n, m)$, regardless of the shape of what follows. The *existential* conjunct $\mathsf{T}(k, nat)$ is different: it is a fact provided to the prover, not an obligation; it may be dropped either as a weakening at a positive position of a premise (Definition 6.7 and Theorem 6.8) or, here, as an equivalence via the dual junk-*falsity* judgment, because the remaining conjunct $le(n, k)$ covers k (Theorem 6.6); the tightness example of Section 8 shows that licensing existential guard omission by the universal (junk-truth) rule instead would be unsound.

A guard that must stay, and one covered by it. The lemma app_nil_r : $\forall A{:}\mathsf{Type}.\ \forall l{:}list\ A.\ l +\!\!+ nil = l$ translates to

$$\forall \alpha.\ \mathsf{T}(\alpha, \mathsf{Type}) \to \forall l.\ \mathsf{T}(l, list(\alpha)) \to app(\alpha, l, nil(\alpha)) = l.$$

The guard on l is *not* omissible: the body is a bare equation, no atom covers l, and dropping the guard yields a sentence that is false in the intended model (the equation fails for junk l, where the left-hand side is a stuck application). The guard on α, however, *is* omissible: the retained guard $\mathsf{T}(l, list(\alpha))$ itself covers it, because typing atoms whose type side is a rigid application of an inductive type former force well-formedness of the former's arguments ($\mathsf{T}(l, list(\alpha))$ can hold only when α denotes a type). Covers may thus come from ordinary hypothesis atoms and from *retained* guards; since a guard that serves as a cover must itself not be dropped, the omissible set is computed as a greatest fixpoint, and Proposition 6.12 shows that any such fixpoint is a sound simultaneous omission.

Type arguments. The constructor $cons : \forall A{:}\mathsf{Type}.\ A \to list\ A \to list\ A$ is translated as a ternary symbol $cons(\alpha, x, l)$. The type argument is recoverable from the declared type of either term argument: A occurs in

the argument types A and *list* A. Erasing it — replacing every occurrence $cons(u, t, s)$ by $cons(t, s)$ uniformly in all premises, axioms and the goal — is sound: in the intended model the omitted argument is a function of the types of the retained ones (Lemma 7.4). By contrast $nil : \forall A{:}\mathsf{Type}.\ list\ A$ has no term argument at all; its type argument is determined only by the *expected* type at each occurrence, which has no counterpart at a first-order term position. Erasing it conflates *nil nat* with *nil bool* as one constant, and Proposition 8.3 turns this into an actual refutation of a goal that fails in a model of the source theory — the formal version of the classic "*card* UNIV" unsoundness of Meng and Paulson [MP08]. Finally, the two optimizations interact: the cover $Even(x)$ forces $x : nat$ only because the predicate symbol carries its full argument list. Erasing type arguments of *predicates* destroys covers, and Proposition 8.4 shows the combination is unsound; the composition theorem (Theorem 7.10) therefore erases only function and constructor symbols.

Proof architecture. All soundness statements are relative to the *ground companion theory* $\mathbb{T}(E)$ of the source environment E: the many-sorted first-order theory of all ground type instances of the premises together with the structural axioms of the inductive definitions. Section 4 defines $\mathbb{T}(E)$ and shows that CIC proves every sentence we place in it, so that a model of $\mathbb{T}(E)$ exists whenever the CIC environment is consistent. Given any model $\mathfrak{N} \models \mathbb{T}(E)$, Section 5 constructs an untyped first-order model $\mathfrak{M}(\mathfrak{N})$ by disjointly uniting (i) the ground types themselves, as denotations of type expressions, (ii) tagged copies $\{(\tau, a) \mid a \in \mathfrak{N}_\tau\}$ of the carriers, and (iii) an inductively generated universe of *junk* elements: formal application trees that record a function symbol applied to arguments violating its type discipline. Junk is what makes the model faithful to the untyped setting — the quantifiers of the translated problem really do range over ill-typed terms — and its *free* (injective, non-confusable) construction is what validates the unguarded injectivity and discrimination axioms that hammer translations emit. Soundness of each optimization then reduces to checking that the transformed axioms remain true in $\mathfrak{M}(\mathfrak{N})$ (or in its erased variant $\mathfrak{M}_\varepsilon(\mathfrak{N})$), and the final theorems all have the same shape: *if the optimized translation of* (E, ϕ_0) *is refutable, then* ϕ_0 *holds in every model of* $\mathbb{T}(E)$.

3 Preliminaries: untyped first-order logic

We fix notation for the target logic and record a replacement lemma used throughout.

Definition 3.1 (Syntax)**.** An (untyped, first-order) *signature* Σ consists of function symbols with arities (constants being 0-ary) and predicate symbols with arities. Terms and formulas over Σ and a countably infinite set of

variables are defined as usual; the connectives are $\bot$, $\top$, $\wedge$, $\vee$, $\to$, and the quantifiers $\forall$, $\exists$; $\neg\phi$ abbreviates $\phi \to \bot$ and $\phi \leftrightarrow \psi$ abbreviates $(\phi \to \psi) \wedge (\psi \to \phi)$. The logic includes a distinguished binary equality predicate $=$. We identify formulas up to renaming of bound variables and assume, whenever a formula is analyzed, that its bound variables are distinct from each other and from all free variables under discussion. $\forall(\phi)$ denotes the universal closure.

Definition 3.2 (Semantics)**.** A Σ-structure $\mathfrak{A}$ consists of a nonempty domain $|\mathfrak{A}|$, functions $h^{\mathfrak{A}} : |\mathfrak{A}|^n \to |\mathfrak{A}|$ for the function symbols and relations $P^{\mathfrak{A}} \subseteq |\mathfrak{A}|^n$ for the predicate symbols; $=$ is always interpreted as identity on $|\mathfrak{A}|$. A *valuation* v maps variables to $|\mathfrak{A}|$; $t^{\mathfrak{A},v}$ and $\mathfrak{A}, v \models \phi$ are defined as usual. For a set Γ of formulas, $\Gamma \models \phi$ means: every structure and valuation satisfying (the universal closures of) all of Γ satisfies $\forall(\phi)$. A set of sentences is *satisfiable* if it has a model and *refutable* if it is unsatisfiable.

Since ATPs are sound and complete for this semantics, "the translated problem is refutable" is interchangeable with "some ATP finds a proof" throughout the paper; none of our results depend on a particular proof calculus.

Definition 3.3 (Polarity)**.** An occurrence of a subformula ψ in a formula χ is *positive* or *negative* according to the following standard recursion: χ is positive in itself; if the occurrence of ψ is in χ_1 or χ_2, its polarity in $\chi_1 \wedge \chi_2$, $\chi_1 \vee \chi_2$, $\forall x.\chi_1$, $\exists x.\chi_1$ equals its polarity in χ_i; its polarity in $\chi_1 \to \chi_2$ equals its polarity in χ_2, and is reversed if the occurrence is in χ_1.

Lemma 3.4 (Replacement)**.** *Let χ' be obtained from χ by replacing one occurrence of a subformula ψ by a formula ψ', where ψ' has no free variables captured differently than in ψ (guaranteed by our convention on bound variables). Let $\mathfrak{A}$ be a structure.*

(i) If $\mathfrak{A} \models \forall(\psi \leftrightarrow \psi')$ then $\mathfrak{A} \models \forall(\chi \leftrightarrow \chi')$.

(ii) If $\mathfrak{A} \models \forall(\psi \to \psi')$ and the occurrence is positive, then $\mathfrak{A} \models \forall(\chi \to \chi')$; if the occurrence is negative, then $\mathfrak{A} \models \forall(\chi' \to \chi)$.

Proof. By induction on the structure of χ. If $\chi = \psi$ is the replaced occurrence, both claims are the assumptions. Otherwise χ is built by a connective or quantifier from immediate subformulas, exactly one of which contains the occurrence; apply the induction hypothesis to it and conclude by the monotonicity of each connective in each argument, for every fixed valuation: $\wedge$, $\vee$, $\forall$, $\exists$ are monotone in each argument, and $\to$ is monotone in its conclusion and antitone in its antecedent — which is precisely how Definition 3.3 propagates polarity. Quantifier cases use the induction hypothesis at all extensions $v[x \mapsto d]$ of the valuation, which is licensed because the premise of the lemma is the universal closure of $\psi \leftrightarrow \psi'$ (respectively $\psi \to \psi'$). $\square$

Remark 3.5. Lemma 3.4 is stated per structure $\mathfrak{A}$; we will apply it with $\mathfrak{A}$ ranging over models of the completion axioms $\mathrm{Comp}(E)$, obtaining entailments of the form $\mathrm{Comp}(E) \models \chi \leftrightarrow \chi'$ from $\mathrm{Comp}(E) \models \psi \leftrightarrow \psi'$.

4 The source fragment PFI

We now define the source calculus: a polymorphic first-order fragment of CIC with parameterized inductive data types and inductive predicates. The fragment is chosen to contain the phenomena that make guard and type-argument omission non-trivial — polymorphism, dependent inductive predicates (predicates whose well-formedness constrains their arguments' types), empty types, equality — while excluding the CIC features (higher-order quantification, dependently typed *data*, conversion) whose treatment by hammer translations involves further, orthogonal machinery; Remark 4.11 delimits the boundary precisely.

4.1 Syntax and typing

Definition 4.1 (Types)**.** Fix a countably infinite set of *type variables* $\alpha, \beta, \ldots$. Given a set $\mathcal{I}$ of *inductive type formers*, each $I \in \mathcal{I}$ with an arity $k_I \geq 0$, the *types over a set V of type variables* are generated by

$$\tau ::= \alpha \mid I(\tau_1, \ldots, \tau_{k_I}), \qquad \alpha \in V,\ I \in \mathcal{I}.$$

We write $\mathrm{Ty}(V)$ for this set, $\mathrm{ftv}(\tau)$ for the type variables occurring in τ, and $\mathrm{GTy} := \mathrm{Ty}(\emptyset)$ for the set of *ground types*. A *type substitution* is a finite map θ from type variables to types, applied in the usual way and written postfix: $\tau\theta$.

Definition 4.2 (Environments)**.** An *environment* E consists of:

(a) a finite set $\mathcal{I}$ of inductive type formers with arities;

(b) a finite set $\mathcal{C}$ of *constructors*; each $c \in \mathcal{C}$ is assigned to exactly one $I \in \mathcal{I}$ and declared as

$$c : \forall \alpha_1, \ldots, \alpha_{k_I}.\, \tau_1^c \times \cdots \times \tau_{m_c}^c \to I(\alpha_1, \ldots, \alpha_{k_I}), \qquad \tau_i^c \in \mathrm{Ty}(\{\alpha_1, \ldots, \alpha_{k_I}\});$$

(c) a finite set $\mathcal{F}$ of *function constants*, each declared as $f : \forall \alpha_1, \ldots, \alpha_{k_f}.\, \tau_1^f \times \cdots \times \tau_{m_f}^f \to \tau_0^f$ with all $\tau_i^f \in \mathrm{Ty}(\{\alpha_1, \ldots, \alpha_{k_f}\})$;

(d) a finite set $\mathcal{P}$ of *predicate constants*, each declared as $p : \forall \alpha_1, \ldots, \alpha_{k_p}.\, \tau_1^p \times \cdots \times \tau_{m_p}^p \to \mathsf{Prop}$ with all $\tau_i^p \in \mathrm{Ty}(\{\alpha_1, \ldots, \alpha_{k_p}\})$;

(e) a finite set $\mathrm{Prem}(E)$ of closed formulas (Definition 4.6), the *premises*.

Zero-ary cases are allowed throughout: I may have no constructors (an empty type), c or f may have no term arguments, and any of the symbol classes may be empty.

Definition 4.3 (Terms and typing)**.** A *context* is a pair $(V;\Gamma)$ of a finite set V of type variables and a finite map Γ from term variables to $\mathrm{Ty}(V)$. Terms are generated by

$$t ::= x \mid h\langle\sigma_1,\dots,\sigma_{k_h}\rangle(t_1,\dots,t_{m_h}), \qquad h \in \mathcal{C}\cup\mathcal{F},$$

where constants are always fully applied to k_h type arguments and m_h term arguments. The typing judgment $V;\Gamma \vdash t : \tau$ (with $\tau \in \mathrm{Ty}(V)$) is defined by two syntax-directed rules:

$$\frac{\Gamma(x) = \tau}{V;\Gamma \vdash x : \tau} \qquad \frac{\sigma_1,\dots,\sigma_{k_h} \in \mathrm{Ty}(V) \qquad V;\Gamma \vdash t_i : \tau_i^h[\bar\sigma/\bar\alpha] \quad (1 \le i \le m_h)}{V;\Gamma \vdash h\langle\bar\sigma\rangle(\bar t) : \tau_0^h[\bar\sigma/\bar\alpha]}$$

where for $h = c \in \mathcal{C}$ assigned to I we set $\tau_0^c := I(\alpha_1,\dots,\alpha_{k_I})$.

Lemma 4.4 (Uniqueness of types)**.** *If $V;\Gamma \vdash t : \tau$ and $V;\Gamma \vdash t : \tau'$ then $\tau = \tau'$.*

Proof. Immediate induction on t: the typing rules are syntax-directed and the type in the conclusion is a function of the syntax ($\Gamma(x)$ for a variable; $\tau_0^h[\bar\sigma/\bar\alpha]$ for an application, where $\bar\sigma$ is written in the term). □

Lemma 4.5 (Stability under instantiation)**.** *Let θ be a type substitution with $\alpha\theta \in \mathrm{Ty}(V')$ for all $\alpha \in V$. If $V;\Gamma \vdash t : \tau$ then $V';\Gamma\theta \vdash t\theta : \tau\theta$, where θ acts on terms by replacing each type-argument tuple $\langle\bar\sigma\rangle$ by $\langle\bar\sigma\theta\rangle$.*

Proof. Induction on the typing derivation, using $\tau_i^h[\bar\sigma/\bar\alpha]\theta = \tau_i^h[\bar\sigma\theta/\bar\alpha]$, which holds because $\mathrm{ftv}(\tau_i^h) \subseteq \{\bar\alpha\}$ and substitution composes. □

Definition 4.6 (Formulas)**.** Formulas and their well-formedness judgment $V;\Gamma \vdash \phi$ wf are generated by:

$$\phi ::= p\langle\bar\sigma\rangle(\bar t) \mid t \approx_\tau s \mid \bot \mid \top \mid \phi\wedge\phi \mid \phi\vee\phi \mid \phi\to\phi \mid \forall x{:}\tau.\ \phi \mid \exists x{:}\tau.\ \phi \mid \forall\alpha.\ \phi$$

with the evident rules: an atom $p\langle\bar\sigma\rangle(\bar t)$ is well formed when $\bar\sigma \in \mathrm{Ty}(V)^{k_p}$ and $V;\Gamma \vdash t_i : \tau_i^p[\bar\sigma/\bar\alpha]$ for all i; an equation $t \approx_\tau s$ when both sides have type $\tau \in \mathrm{Ty}(V)$; quantifiers extend Γ (resp. V). We use $\neg\phi := \phi \to \bot$ and $\leftrightarrow$ as abbreviations. A *sentence* is a formula well formed in the empty context; premises in $\mathrm{Prem}(E)$ are required to be sentences. *Type quantification is prenex*: in $\forall\alpha.\phi$ we require that ϕ contain no term quantifier scoping over a later type quantifier; equivalently, every sentence has the form $\forall\alpha_1,\dots,\alpha_n.\ \psi$ with ψ free of type quantifiers. Lemma 4.5 extends to formulas by the same induction.

4.2 The ground companion theory

The translation's soundness will be stated relative to a many-sorted first-order theory that collects everything CIC proves about PFI-sentences by case analysis on inductive data and inductive proofs. We first fix the many-sorted setting.

Definition 4.7 (Ground signature and structures)**.** The *ground signature* of E has sort set GTy and, for every $h \in \mathcal{C} \cup \mathcal{F}$ and every $\bar{\tau} \in \mathrm{GTy}^{k_h}$, a function symbol $h_{\bar{\tau}}$ of rank $\tau_1^h[\bar{\tau}/\bar{\alpha}] \times \cdots \times \tau_{m_h}^h[\bar{\tau}/\bar{\alpha}] \to \tau_0^h[\bar{\tau}/\bar{\alpha}]$, and for every $p \in \mathcal{P}$, $\bar{\tau} \in \mathrm{GTy}^{k_p}$ a predicate symbol $p_{\bar{\tau}}$ of rank $\tau_1^p[\bar{\tau}/\bar{\alpha}] \times \cdots \times \tau_{m_p}^p[\bar{\tau}/\bar{\alpha}]$. A *ground structure* $\mathfrak{N}$ assigns to every sort $\tau \in \mathrm{GTy}$ a (possibly empty) carrier set $\mathfrak{N}_\tau$, to every $h_{\bar{\tau}}$ a function between the corresponding carriers, and to every $p_{\bar{\tau}}$ a relation; each equality $\approx_\tau$ is interpreted as identity on $\mathfrak{N}_\tau$. Satisfaction $\mathfrak{N}, v \models \phi$ for type-quantifier-free formulas ϕ with a sort-respecting valuation v is defined as usual (a universal (existential) quantifier over an empty carrier is true (false)). For a sentence with prenex type quantifiers we define *ground satisfaction*

$$\mathfrak{N} \models_{\mathrm{g}} \forall \alpha_1, \ldots, \alpha_n.\ \psi \quad \text{iff} \quad \mathfrak{N} \models \psi[\bar{\tau}/\bar{\alpha}] \text{ for every } \bar{\tau} \in \mathrm{GTy}^n,$$

where the ground instance $\psi[\bar{\tau}/\bar{\alpha}]$ is read over the ground signature by interpreting each $h\langle\bar{\sigma}\rangle$ as $h_{\bar{\sigma}[\bar{\tau}/\bar{\alpha}]}$ (well-typedness of instances is Lemma 4.5). We allow empty carriers deliberately: an inductive type former with no constructors, or with constructors whose argument types are empty, denotes an empty collection, and the exhaustiveness axioms below force this.

Definition 4.8 (Structural axioms)**.** Struct(E) is the following set of PFI-sentences. For every $I \in \mathcal{I}$ with constructors $c^{(1)}, \ldots, c^{(r)}$ (possibly $r = 0$) and for $\bar{\alpha}$ of length k_I:

(S1) *Injectivity*, for each constructor $c = c^{(j)}$:

$$\forall \bar{\alpha}.\ \forall \bar{x}{:}\bar{\tau^c}.\ \forall \bar{x}'{:}\bar{\tau^c}.\ c\langle\bar{\alpha}\rangle(\bar{x}) \approx_{I(\bar{\alpha})} c\langle\bar{\alpha}\rangle(\bar{x}') \to \textstyle\bigwedge_i x_i \approx_{\tau_i^c} x_i'.$$

(S2) *Distinctness*, for each pair $j \neq j'$: $\forall \bar{\alpha}\, \bar{x}\, \bar{x}'.\ \neg\big(c^{(j)}\langle\bar{\alpha}\rangle(\bar{x}) \approx_{I(\bar{\alpha})} c^{(j')}\langle\bar{\alpha}\rangle(\bar{x}')\big)$.

(S3) *Exhaustiveness*: $\forall \bar{\alpha}.\ \forall z{:}I(\bar{\alpha}).\ \bigvee_{j=1}^{r} \exists \bar{x}{:}\bar{\tau^{c^{(j)}}}.\ z \approx_{I(\bar{\alpha})} c^{(j)}\langle\bar{\alpha}\rangle(\bar{x})$ (the empty disjunction being $\bot$).

If some predicates of E are designated as *inductively defined* with given clauses, Struct(E) additionally contains their introduction rules and their inversion principles, each of which is a PFI-sentence (for *le*, the inversion principle is the formula displayed in Section 2 before translation). We do not fix a clause format; we only require that whatever is put into Struct(E) for predicates be CIC-provable in the sense of Proposition 4.10.

Definition 4.9 (Ground companion theory)**.** $\mathbb{T}(E)$ is the theory $\mathrm{Prem}(E) \cup \mathrm{Struct}(E)$, considered via ground satisfaction: $\mathfrak{N} \models \mathbb{T}(E)$ means $\mathfrak{N} \models_{\mathrm{g}} \phi$ for every $\phi \in \mathrm{Prem}(E) \cup \mathrm{Struct}(E)$. We write $\mathbb{T}(E) \models_{\mathrm{g}} \phi_0$ to mean that every ground structure satisfying $\mathbb{T}(E)$ ground-satisfies the sentence ϕ_0. E is *consistent* if $\mathbb{T}(E)$ has at least one ground model.

4.3 Relation to CIC

The fragment embeds into CIC in the obvious way: each I becomes a parameterized inductive type in Type, constructors become constructors, function constants become section variables (or definitions) of the corresponding simple type, predicate constants become variables of predicate type or inductive definitions in Prop, formulas become propositions — with $\approx_\tau$ read as Leibniz equality and $\forall\alpha.\phi$ as $\Pi\alpha{:}\mathsf{Type}.\,\phi$ — and premises become hypotheses. Call E *CIC-realizable* if the embedded environment is well formed and every premise's embedding is inhabited.

Proposition 4.10 (CIC bridge)**.** *Assume E is CIC-realizable and the embeddings of the structural axioms (S1)–(S3) and of the predicate introduction/inversion sentences are inhabited — which they are whenever the embedded predicates are inductively defined with the corresponding clauses, since (S1) and (S2) are proved by dependent case analysis and an injection/discrimination argument, (S3) by case analysis on z, and predicate inversion by case analysis on the proof. If CIC extended with excluded middle is consistent, then E is consistent in the sense of Definition 4.9.*

Proof. Interpret the embedded environment in the set-theoretic model of CIC with classical logic [Wer97, LW11] (any model of CIC+EM suffices). Define $\mathfrak{N}$ by letting $\mathfrak{N}_\tau$ be the set interpreting the embedding of the ground type τ, and interpreting each $h_{\bar\tau}$, $p_{\bar\tau}$ by the interpretation of the embedded constant at the corresponding type instance. A straightforward induction on type-quantifier-free PFI-formulas shows that classical set-theoretic truth of the embedding of a ground instance coincides with $\mathfrak{N} \models \cdot$ of that instance: atoms and equations agree by construction, the connectives are interpreted classically on both sides, and term quantifiers range over $\mathfrak{N}_\tau$ on both sides. Every sentence of $\mathbb{T}(E)$ is inhabited in CIC(+EM) by assumption, hence true in the model, hence all its ground instances are true, hence $\mathfrak{N} \models_{\mathrm{g}}$ each of them. Thus $\mathfrak{N} \models \mathbb{T}(E)$. □

From here on we never mention CIC again until Section 9: all theorems are stated for an arbitrary environment E and quantify over ground models $\mathfrak{N} \models \mathbb{T}(E)$. Proposition 4.10 is the (only) bridge: for a CIC-realizable environment, "$\mathbb{T}(E) \models_{\mathrm{g}} \phi_0$" is a sound notion of the goal ϕ_0 being a (classical) consequence of the development, and a conclusion of the form "the optimized translation is refutable $\Rightarrow \mathbb{T}(E) \models_{\mathrm{g}} \phi_0$" is exactly the guarantee

that the ATP has not proved anything beyond what the source environment classically entails.

Remark 4.11 (Scope). Compared to full CIC, PFI excludes: function types as data (arguments and results of constants are inductive types or type variables, so the translation needs no lambda-lifting and no extensionality reasoning); dependently typed data (type formers take only type parameters, not term indices — but *predicates* may have term arguments of arbitrary fragment types, which is where the interesting covers arise); higher-rank polymorphism (type quantification is prenex, as in ML); universes above the single implicit Type; and conversion (constants are opaque; definitional equations enter as premises). These restrictions match the first-order core that hammer translations handle natively; the excluded features are handled by upstream devices (lifting, collapsing) whose interaction with omission we discuss in Section 9. Within the fragment, however, we impose no restriction on the shapes of premises, on emptiness of types, or on which guards and type arguments occur where.

5 The guarded translation and its intended models

This section defines the guarded translation of PFI into untyped FOL, the axiom set it emits, and — the heart of the soundness argument — a family of untyped models validating the emitted axioms.

5.1 Target signature and translation

Definition 5.1 (Target signature)**.** The untyped signature Σ_E has:

- a function symbol $\hat{h}$ of arity $k_h + m_h$ for every $h \in \mathcal{C} \cup \mathcal{F}$;
- a function symbol $\hat{I}$ of arity k_I for every $I \in \mathcal{I}$;
- a constant Type;
- a predicate symbol $\hat{p}$ of arity $k_p + m_p$ for every $p \in \mathcal{P}$, and a binary predicate symbol T.

We reuse PFI's type variables and term variables as FOL variables.

Definition 5.2 (Translation)**.** The term translation $[\![\cdot]\!]$ maps PFI-terms and PFI-types to Σ_E-terms:

$$[\![x]\!] := x, \qquad [\![\alpha]\!] := \alpha, \qquad [\![I(\bar{\sigma})]\!] := \hat{I}([\![\bar{\sigma}]\!]), \qquad [\![h\langle\bar{\sigma}\rangle(\bar{t})]\!] := \hat{h}([\![\bar{\sigma}]\!], [\![\bar{t}]\!]).$$

The formula translation is defined by

$$\begin{aligned} [\![p\langle\bar{\sigma}\rangle(\bar{t})]\!] &:= \hat{p}([\![\bar{\sigma}]\!], [\![\bar{t}]\!]) & [\![t \approx_\tau s]\!] &:= [\![t]\!] = [\![s]\!] \\ [\![\forall x{:}\tau.\ \phi]\!] &:= \forall x.\ \mathsf{T}(x, [\![\tau]\!]) \to [\![\phi]\!] & [\![\exists x{:}\tau.\ \phi]\!] &:= \exists x.\ \mathsf{T}(x, [\![\tau]\!]) \wedge [\![\phi]\!] \\ [\![\forall \alpha.\ \phi]\!] &:= \forall \alpha.\ \mathsf{T}(\alpha, \mathsf{Type}) \to [\![\phi]\!] \end{aligned}$$

and homomorphically on $\bot$, $\top$, $\wedge$, $\vee$, $\to$.

Definition 5.3 (Emitted axioms)**.** The *translation of the environment* is $\Theta(E) := \mathrm{TA} \cup \mathrm{ID} \cup [\![\mathrm{Prem}(E)]\!] \cup [\![\mathrm{Struct}(E)_{\exists}]\!]$, where:

(T1) *Typing axioms* TA: for every $h \in \mathcal{C} \cup \mathcal{F}$,

$$\forall \bar{\alpha}\, \bar{x}.\ \bigwedge_j \mathsf{T}(\alpha_j, \mathsf{Type}) \to \bigwedge_i \mathsf{T}(x_i, [\![\tau_i^h]\!]) \to \mathsf{T}\big(\hat{h}(\bar{\alpha}, \bar{x}),\ [\![\tau_0^h]\!]\big),$$

(iterated implication, associated to the right) and for every $I \in \mathcal{I}$: $\forall \bar{\alpha}.\ \bigwedge_j \mathsf{T}(\alpha_j, \mathsf{Type}) \to \mathsf{T}(\hat{I}(\bar{\alpha}), \mathsf{Type})$.

(T2) *Unguarded injectivity and discrimination* ID: for every constructor c of I,

$$\forall \bar{\alpha}\, \bar{x}\, \bar{\alpha}'\, \bar{x}'.\ \hat{c}(\bar{\alpha}, \bar{x}) = \hat{c}(\bar{\alpha}', \bar{x}') \to \textstyle\bigwedge_j \alpha_j = \alpha_j' \wedge \bigwedge_i x_i = x_i',$$

and for every pair $c \neq c'$ of constructors of the same I: $\forall \bar{\alpha}\, \bar{x}\, \bar{\alpha}'\, \bar{x}'.\ \hat{c}(\bar{\alpha}, \bar{x}) \neq \hat{c}'(\bar{\alpha}', \bar{x}')$. These are deliberately *unguarded* and quantify the two sides' type arguments independently, as hammer translations emit them.

(T3) *Translated premises and structural sentences*: the $[\![\cdot]\!]$-images of all sentences in $\mathrm{Prem}(E)$ and of the exhaustiveness sentences (S3) and predicate introduction/inversion sentences of $\mathrm{Struct}(E)$ (written $\mathrm{Struct}(E)_{\exists}$; the guarded images of (S1)–(S2) are logical consequences of ID and need not be emitted).

Separately, the *typehood-completion axioms* are $\mathrm{Comp}(E) := \mathrm{Cp} \cup \mathrm{Ct}$ with

(C1) Cp: for every $p \in \mathcal{P}$, every $j \leq k_p$ and every $i \leq m_p$,

$$\forall \bar{\alpha}\, \bar{x}.\ \hat{p}(\bar{\alpha}, \bar{x}) \to \mathsf{T}(\alpha_j, \mathsf{Type}), \qquad \forall \bar{\alpha}\, \bar{x}.\ \hat{p}(\bar{\alpha}, \bar{x}) \to \mathsf{T}(x_i, [\![\tau_i^p]\!]);$$

(C2) Ct: for every $I \in \mathcal{I}$ and $j \leq k_I$: $\forall \bar{\alpha}\, z.\ \mathsf{T}(z, \hat{I}(\bar{\alpha})) \to \mathsf{T}(\alpha_j, \mathsf{Type})$.

Given a goal sentence ϕ_0, the *translated problem* is $\Theta(E) \cup \mathrm{Comp}(E) \cup \{\neg [\![\phi_0]\!]\}$.

5.2 The intended models $\mathfrak{M}(\mathfrak{N})$

Fix a ground structure $\mathfrak{N} \models \mathbb{T}(E)$. We build an untyped Σ_E-structure whose domain contains, besides denotations for all ground types and all their inhabitants, explicit *junk* elements representing ill-typed applications. Junk is generated *freely*, which is what validates the unguarded axioms ID.

Definition 5.4 (Domain)**.** Let

$$D_0 := \underbrace{\mathrm{GTy}}_{\text{type elements}} \uplus \underbrace{\{*\}}_{\text{the element } \mathsf{Type}} \uplus \underbrace{\{(\tau, a) \mid \tau \in \mathrm{GTy},\ a \in \mathfrak{N}_\tau\}}_{\text{tagged carrier elements}}.$$

For a function symbol $g \in \Sigma_E$ of arity n and a tuple $\bar{d} \in (D_0 \uplus J)^n$ define the *sort check* $\mathrm{ok}_g(\bar{d})$:

- $\mathrm{ok}_{\hat{I}}(\bar{d})$ iff $d_j \in \mathrm{GTy}$ for all $j \le k_I$;
- $\mathrm{ok}_{\hat{h}}(\bar{d})$ (for $h \in \mathcal{C} \cup \mathcal{F}$) iff $d_j \in \mathrm{GTy}$ for $j \le k_h$ — write $\tau_j := d_j$ — and $d_{k_h+i} = \big(\tau_i^h[\bar{\tau}/\bar{\alpha}],\, a_i\big)$ for some $a_i \in \mathfrak{N}_{\tau_i^h[\bar{\tau}/\bar{\alpha}]}$, for all $i \le m_h$;
- $\mathrm{ok}_{\mathsf{Type}}()$ always.

The junk universe J is the least set of formal pairs closed under: if $g \in \Sigma_E$ is a function symbol of arity $n \ge 1$, $\bar{d} \in (D_0 \uplus J)^n$ and $\neg\mathrm{ok}_g(\bar{d})$, then $(g, \bar{d}) \in J$. (Formally, $J = \bigcup_n J_n$ with $J_0 = \emptyset$ and J_{n+1} the set of such pairs over $D_0 \uplus J_n$; the union is closed because ok_g never holds when some argument lies in J.) The four classes of elements — ground types, $*$, tagged pairs, junk pairs — are pairwise disjoint (formally, tag them). Finally $D := D_0 \uplus J$; note $D \neq \emptyset$ since $* \in D$.

Definition 5.5 (The structure $\mathfrak{M}(\mathfrak{N})$)**.** $\mathfrak{M}(\mathfrak{N})$ is the Σ_E-structure with domain D and:

- $\mathsf{Type}^{\mathfrak{M}(\mathfrak{N})} := *$;
- $\hat{I}^{\mathfrak{M}(\mathfrak{N})}(\bar{d}) := I(\bar{d}) \in \mathrm{GTy}$ if $\mathrm{ok}_{\hat{I}}(\bar{d})$, and $(\hat{I}, \bar{d}) \in J$ otherwise;
- for $h \in \mathcal{C} \cup \mathcal{F}$: $\hat{h}^{\mathfrak{M}(\mathfrak{N})}(\bar{d}) := \big(\tau_0^h[\bar{\tau}/\bar{\alpha}],\ h^{\mathfrak{N}}_{\bar{\tau}}(\bar{a})\big)$ if $\mathrm{ok}_{\hat{h}}(\bar{d})$ holds with $\bar{\tau}, \bar{a}$ as in Definition 5.4, and $(\hat{h}, \bar{d}) \in J$ otherwise;
- $\mathsf{T}^{\mathfrak{M}(\mathfrak{N})}(d, u)$ iff either $u = *$ and $d \in \mathrm{GTy}$, or $u \in \mathrm{GTy}$ and $d = (u, a)$ for some $a \in \mathfrak{N}_u$;
- $\hat{p}^{\mathfrak{M}(\mathfrak{N})}(\bar{d})$ iff ok holds in the evident sense — $d_j \in \mathrm{GTy}$ for $j \le k_p$ (write $\bar{\tau}$) and $d_{k_p+i} = (\tau_i^p[\bar{\tau}/\bar{\alpha}], a_i)$ — *and* $p^{\mathfrak{N}}_{\bar{\tau}}(\bar{a})$ holds.

All interpretations are total and well defined: the sort check determines $\bar{\tau}$ and $\bar{a}$ uniquely (tags are part of the elements), and the junk branch applies exactly when it fails.

Note the two design decisions embodied in Definition 5.5. First, $\hat{p}^{\mathfrak{M}(\mathfrak{N})}$ is *false-extended*: it fails on every argument tuple violating p's type discipline. This is the semantic core of guard omission — it is the model-theoretic counterpart of "an ill-typed application is not a provable proposition" — and it makes the completion axioms true (Theorem 5.8). Second, junk is

free: an ill-typed application denotes the formal tree recording its head and arguments, so distinct ill-typed applications denote distinct elements. This validates the unguarded injectivity and discrimination axioms, giving a precise sense to the folklore justification that they are "true in the term model".

5.3 Denotation and relativization

Throughout, let ϕ, t, τ be well formed in a context $(V;\Gamma)$, let $\theta : V \to \mathrm{GTy}$ be a ground type instantiation, and let v be a *proper valuation*: a sort-respecting assignment $v(x) \in \mathfrak{N}_{\Gamma(x)\theta}$ for term variables. Define the FOL valuation $w_{\theta,v}$ on $\mathfrak{M}(\mathfrak{N})$ by $w_{\theta,v}(\alpha) := \theta(\alpha)$ for $\alpha \in V$ and $w_{\theta,v}(x) := (\Gamma(x)\theta,\, v(x))$.

Lemma 5.6 (Denotation)**.** *For every type $\sigma \in \mathrm{Ty}(V)$: $[\![\sigma]\!]^{\mathfrak{M}(\mathfrak{N}),w_{\theta,v}} = \sigma\theta \in \mathrm{GTy}$. For every term t with $V;\Gamma \vdash t : \tau$:*

$$[\![t]\!]^{\mathfrak{M}(\mathfrak{N}),w_{\theta,v}} = \big(\tau\theta,\ t\theta^{\mathfrak{N},v}\big),$$

where $t\theta^{\mathfrak{N},v}$ is the many-sorted denotation of the ground instance $t\theta$ (Definition 4.7).

Proof. Both claims by induction on the syntax. Types: a variable α denotes $\theta(\alpha)$; for $I(\bar\sigma)$ the induction hypothesis gives arguments in GTy, so $\mathrm{ok}_{\hat I}$ holds and the value is $I(\bar\sigma\theta) = I(\bar\sigma)\theta$. Terms: a variable x denotes $w_{\theta,v}(x) = (\Gamma(x)\theta, v(x))$ and $\tau = \Gamma(x)$ by the typing rule. For $t = h\langle\bar\sigma\rangle(\bar t)$ with $V;\Gamma \vdash t_i : \tau_i^h[\bar\sigma/\bar\alpha]$: by the induction hypotheses the type arguments denote $\bar\sigma\theta \in \mathrm{GTy}^{k_h}$ and the term arguments denote $\big(\tau_i^h[\bar\sigma/\bar\alpha]\theta,\, t_i\theta^{\mathfrak{N},v}\big) = \big(\tau_i^h[\bar\sigma\theta/\bar\alpha],\, t_i\theta^{\mathfrak{N},v}\big)$ (substitution composition, as in Lemma 4.5). Hence $\mathrm{ok}_{\hat h}$ holds with $\bar\tau = \bar\sigma\theta$, and the value is $\big(\tau_0^h[\bar\sigma\theta/\bar\alpha],\, h^{\mathfrak{N}}_{\bar\sigma\theta}(\dots)\big)$, which is exactly $(\tau\theta, t\theta^{\mathfrak{N},v})$ by the definition of many-sorted denotation of the instance. □

Lemma 5.7 (Relativization)**.** *For every type-quantifier-free formula ϕ well formed in $(V;\Gamma)$, every θ and proper v as above:*

$$\mathfrak{M}(\mathfrak{N}), w_{\theta,v} \models [\![\phi]\!] \quad \textit{iff} \quad \mathfrak{N}, v \models \phi\theta.$$

Consequently, for every sentence ϕ_0 (with prenex type quantifiers): $\mathfrak{M}(\mathfrak{N}) \models [\![\phi_0]\!]$ iff $\mathfrak{N} \models_{\mathrm{g}} \phi_0$.

Proof. Induction on ϕ.

Atoms. $[\![p\langle\bar\sigma\rangle(\bar t)]\!] = \hat p([\![\bar\sigma]\!], [\![\bar t]\!])$. By Lemma 5.6 the arguments denote $\bar\sigma\theta \in \mathrm{GTy}$ and the tagged pairs $(\tau_i^p[\bar\sigma\theta/\bar\alpha], t_i\theta^{\mathfrak{N},v})$, so ok holds and, by Definition 5.5, the atom is true in $\mathfrak{M}(\mathfrak{N})$ iff $p^{\mathfrak{N}}_{\bar\sigma\theta}(\dots)$ holds, i.e. iff $\mathfrak{N}, v \models (p\langle\bar\sigma\rangle(\bar t))\theta$.

Equations. $[\![t]\!] = [\![s]\!]$ holds iff the two tagged pairs are equal; since both carry the same tag $\tau\theta$ (Lemma 5.6, using that t and s have the same type τ), this is equivalent to $t\theta^{\mathfrak{N},v} = s\theta^{\mathfrak{N},v}$, i.e. to $\mathfrak{N}, v \models (t \approx_\tau s)\theta$.

Connectives. Immediate from the induction hypotheses, as both semantics interpret them classically.

Term quantifiers. Consider $[\![\forall x{:}\tau.\ \phi]\!] = \forall x.\ \mathsf{T}(x, [\![\tau]\!]) \to [\![\phi]\!]$. By Lemma 5.6, $[\![\tau]\!]^{\mathfrak{M}(\mathfrak{N}), w_{\theta,v}} = \tau\theta \in \mathrm{GTy}$, so for $d \in D$ the guard $\mathsf{T}(d, \tau\theta)$ holds iff $d = (\tau\theta, a)$ with $a \in \mathfrak{N}_{\tau\theta}$. Hence the $\mathfrak{M}(\mathfrak{N})$-side quantification, restricted by the guard, ranges exactly over $\{(\tau\theta, a) \mid a \in \mathfrak{N}_{\tau\theta}\}$; for each such d the extended valuation is $w_{\theta,v[x\mapsto a]}$ and the induction hypothesis applies. This matches the $\mathfrak{N}$-side clause "for all $a \in \mathfrak{N}_{\tau\theta}$", including the empty-carrier case (both sides vacuously true). The existential case is dual (both sides false over an empty carrier).

Sentences. For $\phi_0 = \forall\alpha_1, \ldots, \alpha_n.\psi$: $[\![\phi_0]\!] = \forall\bar{\alpha}.\ \mathsf{T}(\alpha_1, \mathsf{Type}) \to \ldots$, and $\mathsf{T}^{\mathfrak{M}(\mathfrak{N})}(d, *)$ holds iff $d \in \mathrm{GTy}$. So $\mathfrak{M}(\mathfrak{N}) \models [\![\phi_0]\!]$ iff $\mathfrak{M}(\mathfrak{N}), [\bar{\alpha} \mapsto \bar{\tau}] \models [\![\psi]\!]$ for all $\bar{\tau} \in \mathrm{GTy}^n$, which by the main claim (with $\theta = [\bar{\tau}/\bar{\alpha}]$, empty Γ-part) is equivalent to $\mathfrak{N} \models \psi[\bar{\tau}/\bar{\alpha}]$ for all $\bar{\tau}$, i.e. to $\mathfrak{N} \models_{\mathrm{g}} \phi_0$. □

5.4 Base soundness

Theorem 5.8 (Model correctness)**.** *For every ground structure $\mathfrak{N} \models \mathbb{T}(E)$: $\mathfrak{M}(\mathfrak{N}) \models \Theta(E) \cup \mathrm{Comp}(E)$.*

Proof. We check each axiom class against Definition 5.5; fix an arbitrary valuation w into D.

(T1) Typing axioms. Assume the guards: $w(\alpha_j) \in \mathrm{GTy}$ for all j (so they determine θ) and $\mathsf{T}(w(x_i), [\![\tau_i^h]\!]^w)$ for all i. Since $w(\bar{\alpha}) = \bar{\tau} \in \mathrm{GTy}$, the first part of Lemma 5.6 gives $[\![\tau_i^h]\!]^w = \tau_i^h[\bar{\tau}/\bar{\alpha}]$ (the lemma's proof uses only the values of w on $\mathrm{ftv}(\tau_i^h)$, all in GTy). So each $w(x_i)$ is a tagged pair $(\tau_i^h[\bar{\tau}/\bar{\alpha}], a_i)$; thus $\mathrm{ok}_{\hat{h}}$ holds, the value of $\hat{h}(\bar{\alpha}, \bar{x})$ under w is $(\tau_0^h[\bar{\tau}/\bar{\alpha}], h_{\bar{\tau}}^{\mathfrak{N}}(\bar{a}))$, and the conclusion T-atom holds. The $\hat{I}$ axioms are analogous ($I(\bar{\tau}) \in \mathrm{GTy}$ and $\mathsf{T}(I(\bar{\tau}), *)$ holds).

(T2) Injectivity. Let $d := \hat{c}^{\mathfrak{M}(\mathfrak{N})}(w(\bar{\alpha}), w(\bar{x}))$ and $d' := \hat{c}^{\mathfrak{M}(\mathfrak{N})}(w(\bar{\alpha}'), w(\bar{x}'))$ and assume $d = d'$. Three cases. (i) Both sort checks succeed: then $d = (I(\bar{\tau}), c_{\bar{\tau}}^{\mathfrak{N}}(\bar{a}))$ and $d' = (I(\bar{\tau}'), c_{\bar{\tau}'}^{\mathfrak{N}}(\bar{a}'))$; equality of the tags gives $I(\bar{\tau}) = I(\bar{\tau}')$ in GTy, whence $\bar{\tau} = \bar{\tau}'$ because ground types are syntax (a first-order term algebra, in which constructors are injective); then $c_{\bar{\tau}}^{\mathfrak{N}}(\bar{a}) = c_{\bar{\tau}}^{\mathfrak{N}}(\bar{a}')$, and $\mathfrak{N} \models_{\mathrm{g}}$ (S1) at instance $\bar{\tau}$ yields $\bar{a} = \bar{a}'$; hence $w(\alpha_j) = \tau_j = \tau_j' = w(\alpha_j')$ and $w(x_i) = (\tau_i^c[\bar{\tau}/\bar{\alpha}], a_i) = w(x_i')$. (ii) Both fail: $d = (\hat{c}, w(\bar{\alpha})w(\bar{x}))$ and $d' = (\hat{c}, w(\bar{\alpha}')w(\bar{x}'))$ are formal pairs; their equality gives componentwise equality of the argument tuples, which is the conclusion. (iii) Exactly one fails: then d is a tagged pair and d' a junk pair (or vice versa), contradicting $d = d'$; the implication holds vacuously.

(T2) Discrimination. With d, d' as above for $c \neq c'$ of the same I: if both sort checks succeed and the tags agree, then $\bar{\tau} = \bar{\tau}'$ as before and $\mathfrak{N} \models_{\mathrm{g}}$ (S2) at $\bar{\tau}$ gives $c_{\bar{\tau}}^{\mathfrak{N}}(\bar{a}) \neq c_{\bar{\tau}}'^{\mathfrak{N}}(\bar{a}')$; if the tags differ, $d \neq d'$ outright. If both fail, the junk pairs have distinct heads $\hat{c} \neq \hat{c}'$. Mixed cases are distinct by

disjointness.

(T3) Translated sentences. Each is $[\![\phi]\!]$ for a sentence ϕ with $\mathfrak{N} \models_{\mathrm{g}} \phi$ (premises and the retained structural sentences are in $\mathbb{T}(E)$); apply Lemma 5.7.

(C1) Predicate completion. If $\hat{p}^{\mathfrak{M}(\mathfrak{N})}(w(\bar{\alpha}), w(\bar{x}))$ holds then, by Definition 5.5, $w(\alpha_j) \in \mathrm{GTy}$ — so $\mathsf{T}(w(\alpha_j), *)$ holds — and $w(x_i) = (\tau_i^p[\bar{\tau}/\bar{\alpha}], a_i)$; since $[\![\tau_i^p]\!]^w = \tau_i^p[\bar{\tau}/\bar{\alpha}]$ (Lemma 5.6, as in (T1)), the atom $\mathsf{T}(x_i, [\![\tau_i^p]\!])$ holds.

(C2) Hereditary completion. Assume $\mathsf{T}(w(z), \hat{I}(\bar{\alpha})^w)$. By Definition 5.5 the second argument must lie in $\mathrm{GTy} \cup \{*\}$; the value $\hat{I}^{\mathfrak{M}(\mathfrak{N})}(w(\bar{\alpha}))$ lies in GTy only if $\mathrm{ok}_{\hat{I}}$ holds, i.e. all $w(\alpha_j) \in \mathrm{GTy}$ (a junk pair is neither a ground type nor $*$). In that case $\mathsf{T}(w(\alpha_j), *)$ holds for each j; otherwise the assumption is false and the implication vacuous. □

Theorem 5.9 (Soundness of the guarded translation)**.** *Let E be an environment and ϕ_0 a sentence. If $\Theta(E) \cup \mathrm{Comp}(E) \cup \{\neg[\![\phi_0]\!]\}$ is unsatisfiable, then $\mathbb{T}(E) \models_{\mathrm{g}} \phi_0$. In particular, if E is consistent then $\Theta(E) \cup \mathrm{Comp}(E)$ is satisfiable.*

Proof. Contrapositive. If $\mathbb{T}(E) \not\models_{\mathrm{g}} \phi_0$, pick a ground structure $\mathfrak{N} \models \mathbb{T}(E)$ with $\mathfrak{N} \not\models_{\mathrm{g}} \phi_0$. By Theorem 5.8, $\mathfrak{M}(\mathfrak{N}) \models \Theta(E) \cup \mathrm{Comp}(E)$, and by Lemma 5.7, $\mathfrak{M}(\mathfrak{N}) \not\models [\![\phi_0]\!]$, i.e. $\mathfrak{M}(\mathfrak{N}) \models \neg[\![\phi_0]\!]$. So the translated problem is satisfiable. The second claim takes any $\mathfrak{N} \models \mathbb{T}(E)$ and observes $\mathfrak{M}(\mathfrak{N}) \models \Theta(E) \cup \mathrm{Comp}(E)$. □

Remark 5.10. Theorem 5.9 quantifies over *all* models of $\mathbb{T}(E)$, so its conclusion $\mathbb{T}(E) \models_{\mathrm{g}} \phi_0$ is classical semantic entailment from the source theory — via Proposition 4.10, a consequence of the CIC environment under excluded middle. This is the standard guarantee for hammers, whose final arbiter is proof reconstruction in the proof assistant; what the theorem rules out is the ATP refuting a translation of a goal that the source theory does not entail. All optimization theorems below preserve exactly this guarantee.

6 Guard omission

We now formulate the guard-omission criterion and prove it sound. The section is purely first-order: all results hold in an arbitrary Σ_E-structure satisfying $\mathrm{Comp}(E)$, and the intended models enter only through $\mathfrak{M}(\mathfrak{N}) \models \mathrm{Comp}(E)$ (Theorem 5.8). Throughout, "formula" means Σ_E-formula; the results apply to the images of the translation but are not restricted to them.

6.1 Typing consequences and covers

Definition 6.1 (Typing consequences)**.** For a Σ_E-atom A, the set $\mathrm{tc}(A)$ of *typing consequences* of A is defined by:

$$\begin{aligned}\mathrm{tc}(\hat{p}(\bar{s},\bar{t})) &:= \{\,\mathsf{T}(s_j,\mathsf{Type}) \mid 1\le j\le k_p\,\}\cup\{\,\mathsf{T}(t_i,\ [\![\tau_i^p]\!][\bar{s}/\bar{\alpha}]) \mid 1\le i\le m_p\,\},\\ \mathrm{tc}(\mathsf{T}(s,\hat{I}(\bar{s}))) &:= \{\,\mathsf{T}(s_j,\mathsf{Type}) \mid 1\le j\le k_I\,\},\\ \mathrm{tc}(A) &:= \emptyset \quad \text{for every other atom}\end{aligned}$$

— in particular $\mathrm{tc}(A)=\emptyset$ for equations $s=t$, for T-atoms whose second argument is a variable or the constant Type, and for any atom not of the two shapes above. Here $[\![\tau_i^p]\!][\bar{s}/\bar{\alpha}]$ is the Σ_E-term obtained by substituting the argument terms $\bar{s}$ for the type variables $\bar{\alpha}$ in the translated declared argument type; note $[\![\tau_i^p]\!]$ contains no variables besides $\bar{\alpha}$.

Lemma 6.2 (Strictness)**.** *For every atom A and every $B\in\mathrm{tc}(A)$:* $\mathrm{Comp}(E)\models\forall(A\to B)$.

Proof. Let $\mathfrak{A}\models\mathrm{Comp}(E)$ and let w be a valuation with $\mathfrak{A},w\models A$. If $A=\hat{p}(\bar{s},\bar{t})$ and $B=\mathsf{T}(t_i,[\![\tau_i^p]\!][\bar{s}/\bar{\alpha}])$, consider the axiom $\forall\bar{\alpha}\,\bar{x}.\ \hat{p}(\bar{\alpha},\bar{x})\to\mathsf{T}(x_i,[\![\tau_i^p]\!])$ of Cp. Applying it at the valuation sending $\alpha_j\mapsto s_j^{\mathfrak{A},w}$ and $x_i\mapsto t_i^{\mathfrak{A},w}$, and using the standard substitution property $r[\bar{s}/\bar{\alpha}]^{\mathfrak{A},w}=r^{\mathfrak{A},w[\bar{\alpha}\mapsto\bar{s}^{\mathfrak{A},w}]}$ of first-order semantics, we obtain $\mathfrak{A},w\models B$. The cases $B=\mathsf{T}(s_j,\mathsf{Type})$ and $A=\mathsf{T}(s,\hat{I}(\bar{s}))$ are identical, using the other Cp axioms and Ct respectively. □

Definition 6.3 (Cover)**.** Let x be a variable and u a Σ_E-term with $x\notin\mathrm{fv}(u)$. An atom A *covers* (x,u) if $\mathsf{T}(x,u)\in\mathrm{tc}(A)$ (syntactic identity of terms).

Thus $\mathit{Even}(x)$ covers (x,nat); $\hat{le}(n,m)$ covers (n,nat) and (m,nat); $\mathsf{T}(l,\widehat{\mathit{list}}(\alpha))$ covers (α,Type); an equation covers nothing; $\mathsf{T}(x,\beta)$ with variable β covers nothing. The exclusions are not incidental — each is witnessed by a counterexample in Section 8 or by failure of Lemma 6.2 in $\mathfrak{M}(\mathfrak{N})$ (e.g. $\mathsf{T}^{\mathfrak{M}(\mathfrak{N})}(d,\beta)$ can hold with β valued $*$, in which case $\mathsf{T}(\beta,\mathsf{Type})$ fails).

6.2 Junk-truth and junk-falsity

Definition 6.4 (jt/jf)**.** Fix (x,u) as in Definition 6.3. The judgments $\mathrm{jt}_{x,u}(\phi)$ (*junk-true*) and $\mathrm{jf}_{x,u}(\phi)$ (*junk-false*) are the least pair of predicates on formulas closed under the following rules, where "A covers" abbreviates "A is an atom covering (x,u)", and bound variables are (by our convention)

distinct from x and from the variables of u:

$\mathrm{jt}(\top)$	$\mathrm{jf}(\bot)$
$\mathrm{jt}(A \to \psi)$ if A covers	$\mathrm{jf}(A)$ if A covers
$\mathrm{jt}(\phi_1 \to \phi_2)$ if $\mathrm{jt}(\phi_2)$ or $\mathrm{jf}(\phi_1)$	$\mathrm{jf}(\phi_1 \to \phi_2)$ if $\mathrm{jt}(\phi_1)$ and $\mathrm{jf}(\phi_2)$
$\mathrm{jt}(\phi_1 \wedge \phi_2)$ if $\mathrm{jt}(\phi_1)$ and $\mathrm{jt}(\phi_2)$	$\mathrm{jf}(\phi_1 \wedge \phi_2)$ if $\mathrm{jf}(\phi_1)$ or $\mathrm{jf}(\phi_2)$
$\mathrm{jt}(\phi_1 \vee \phi_2)$ if $\mathrm{jt}(\phi_1)$ or $\mathrm{jt}(\phi_2)$	$\mathrm{jf}(\phi_1 \vee \phi_2)$ if $\mathrm{jf}(\phi_1)$ and $\mathrm{jf}(\phi_2)$
$\mathrm{jt}(\forall y.\psi)$, $\mathrm{jt}(\exists y.\psi)$ if $\mathrm{jt}(\psi)$	$\mathrm{jf}(\forall y.\psi)$, $\mathrm{jf}(\exists y.\psi)$ if $\mathrm{jf}(\psi)$

The first jt-implication rule is the *cover rule*; note that it ignores ψ entirely. Since $\neg\psi = \psi \to \bot$, the derived rules $\mathrm{jt}(\neg\psi) \Leftarrow \mathrm{jf}(\psi)$ and $\mathrm{jt}(\neg A)$ for covering A, as well as $\mathrm{jt}(A \leftrightarrow A')$ for covering A, A', are instances.

Lemma 6.5 (Junk instantiation)**.** *Let $\mathfrak{A} \models \mathrm{Comp}(E)$ and let w be a valuation with $\mathfrak{A}, w \not\models \mathsf{T}(x,u)$. Then $\mathrm{jt}_{x,u}(\phi)$ implies $\mathfrak{A}, w \models \phi$, and $\mathrm{jf}_{x,u}(\phi)$ implies $\mathfrak{A}, w \not\models \phi$.*

Proof. Simultaneous induction on the derivation of the judgment.

Cover cases. If A covers (x,u) then $\mathrm{Comp}(E) \models \forall(A \to \mathsf{T}(x,u))$ by Lemma 6.2, so from $\mathfrak{A}, w \not\models \mathsf{T}(x,u)$ we get $\mathfrak{A}, w \not\models A$. This gives the $\mathrm{jf}(A)$ case directly, and makes $A \to \psi$ true under w, giving the jt cover rule.

Propositional cases. Immediate from the induction hypotheses and the truth tables; e.g. for $\mathrm{jf}(\phi_1 \to \phi_2)$ from $\mathrm{jt}(\phi_1)$ and $\mathrm{jf}(\phi_2)$: by induction $\mathfrak{A}, w \models \phi_1$ and $\mathfrak{A}, w \not\models \phi_2$, so $\mathfrak{A}, w \not\models \phi_1 \to \phi_2$.

Quantifier cases. For $\mathrm{jt}(\forall y.\psi)$ from $\mathrm{jt}(\psi)$: for every $d \in |\mathfrak{A}|$ the valuation $w[y \mapsto d]$ still falsifies $\mathsf{T}(x,u)$, because $y \neq x$ and $y \notin \mathrm{fv}(u)$, so the induction hypothesis gives $\mathfrak{A}, w[y \mapsto d] \models \psi$. For $\mathrm{jt}(\exists y.\psi)$, pick any $d \in |\mathfrak{A}|$ (domains are nonempty) and argue likewise. The jf cases are dual, with $\mathrm{jf}(\forall y.\psi)$ using nonemptiness. □

6.3 The omission theorem

Theorem 6.6 (Guard omission)**.** *Let $x \notin \mathrm{fv}(u)$.*

(i) If $\mathrm{jt}_{x,u}(\phi)$ then $\mathrm{Comp}(E) \models (\forall x.\ \mathsf{T}(x,u) \to \phi) \leftrightarrow (\forall x.\ \phi)$.

(ii) If $\mathrm{jf}_{x,u}(\phi)$ then $\mathrm{Comp}(E) \models (\exists x.\ \mathsf{T}(x,u) \wedge \phi) \leftrightarrow (\exists x.\ \phi)$.

Proof. Let $\mathfrak{A} \models \mathrm{Comp}(E)$ and w be a valuation (of the remaining free variables).

(i) The direction $\Leftarrow$ is weakening of the body and holds outright. For $\Rightarrow$, assume $\mathfrak{A}, w \models \forall x.\ \mathsf{T}(x,u) \to \phi$ and let $d \in |\mathfrak{A}|$. If $\mathfrak{A}, w[x \mapsto d] \models \mathsf{T}(x,u)$, the assumption yields ϕ. Otherwise Lemma 6.5 yields $\mathfrak{A}, w[x \mapsto d] \models \phi$ from $\mathrm{jt}(\phi)$. (The valuation of u is unaffected by $x \mapsto d$ since $x \notin \mathrm{fv}(u)$.)

(ii) The direction $\Rightarrow$ is weakening. For $\Leftarrow$, let d witness $\exists x.\phi$, i.e. $\mathfrak{A}, w[x \mapsto d] \models \phi$. If $\mathfrak{A}, w[x \mapsto d] \not\models \mathsf{T}(x,u)$ then Lemma 6.5 (the jf half) gives

$\mathfrak{A}, w[x \mapsto d] \not\models \phi$, a contradiction; hence $\mathsf{T}(x,u)$ holds at d and d witnesses the guarded existential. □

Both statements are *equivalences relative to* $\mathrm{Comp}(E)$; by the replacement lemma they may be applied to any subformula occurrence, at any polarity, of any premise or of the (negated) goal. We package this, together with the polarity-restricted weakenings for guards that fail the criterion, into a single admissibility notion.

Definition 6.7 (Admissible optimization)**.** A *guard-rewrite step* on a formula χ replaces one occurrence of a subformula

$$\forall x.\, \mathsf{T}(x,u) \to \phi \quad \text{by} \quad \forall x.\, \phi \qquad \text{or} \qquad \exists x.\, \mathsf{T}(x,u) \wedge \phi \quad \text{by} \quad \exists x.\, \phi,$$

and it is *licensed* if $\mathrm{jt}_{x,u}(\phi)$ (universal case) or $\mathrm{jf}_{x,u}(\phi)$ (existential case). A *weakening step* replaces a positive occurrence of $\mathsf{T}(t,u) \wedge \phi$ by ϕ, or of $\forall x.\, \phi$ by $\forall x.\, \mathsf{T}(x,u) \to \phi$, or a negative occurrence dually. An *optimized problem* for (E, ϕ_0) is any set $\Theta' \cup \mathrm{Comp}(E) \cup \{\neg G'\}$ obtained from $\Theta(E) \cup \mathrm{Comp}(E) \cup \{\neg[\![\phi_0]\!]\}$ by finitely many steps, each of which is: (a) a licensed guard-rewrite step applied anywhere in a premise or in G (G initially $[\![\phi_0]\!]$); or (b) a weakening step applied in a premise; or (c) a *strengthening* of the goal: replacement of G by any G'' with $\mathrm{Comp}(E) \models G'' \to G$. The axioms $\mathrm{Comp}(E)$ themselves are never rewritten.

Theorem 6.8 (Soundness of guard omission)**.** *If some optimized problem for (E, ϕ_0) is unsatisfiable, then $\mathbb{T}(E) \models_{\mathrm{g}} \phi_0$.*

Proof. As in Theorem 5.9, assume $\mathbb{T}(E) \not\models_{\mathrm{g}} \phi_0$ and pick $\mathfrak{N} \models \mathbb{T}(E)$ with $\mathfrak{N} \not\models_{\mathrm{g}} \phi_0$; we show $\mathfrak{M}(\mathfrak{N})$ satisfies the optimized problem. $\mathfrak{M}(\mathfrak{N}) \models \mathrm{Comp}(E)$ by Theorem 5.8. By induction on the sequence of steps we show that $\mathfrak{M}(\mathfrak{N})$ satisfies every current premise and the negation of the current goal. Initially this is Theorem 5.8 and Lemma 5.7. For a licensed guard-rewrite step, Theorem 6.6 gives $\mathrm{Comp}(E) \models \psi \leftrightarrow \psi'$ for the replaced subformula, hence (since $\mathfrak{M}(\mathfrak{N}) \models \mathrm{Comp}(E)$) Lemma 3.4(i) preserves the truth value of the rewritten premise, and preserves the truth value of G, hence of $\neg G$. For a weakening step in a premise, $\mathrm{Comp}(E) \models \forall(\psi \to \psi')$ holds for the replaced subformula by pure logic ($\mathsf{T}(t,u) \wedge \phi \models \phi$ and $\forall x.\phi \models \forall x.\mathsf{T}(x,u) \to \phi$), and Lemma 3.4(ii) with the stated polarities gives that the new premise is $\mathrm{Comp}(E)$-entailed by the old one, hence true in $\mathfrak{M}(\mathfrak{N})$. For a goal strengthening, $\mathrm{Comp}(E) \models G'' \to G$ and $\mathfrak{M}(\mathfrak{N}) \models \neg G$ give $\mathfrak{M}(\mathfrak{N}) \models \neg G''$. □

Corollary 6.9 (Conservativity)**.** *Let the optimized problem be obtained using only licensed guard-rewrite steps (no weakenings or strengthenings). Then the optimized problem is satisfiable if and only if the original problem $\Theta(E) \cup \mathrm{Comp}(E) \cup \{\neg[\![\phi_0]\!]\}$ is. In particular, guard omission with completion axioms present loses no ATP proofs.*

Proof. Both problems contain $\mathrm{Comp}(E)$. In any structure $\mathfrak{A} \models \mathrm{Comp}(E)$, each licensed rewrite preserves truth values of the rewritten sentences (Theorem 6.6 with Lemma 3.4(i)), so $\mathfrak{A}$ satisfies the original problem iff it satisfies the optimized one. A model of either is a model of $\mathrm{Comp}(E)$, so the two problems have the same models. □

Remark 6.10. Corollary 6.9 is the formal content of the slogan "drop obligations, keep facts — or emit completion axioms". Without $\mathrm{Comp}(E)$ in the problem, omission remains *sound* (Theorem 6.8 never uses $\mathrm{Comp}(E) \subseteq$ problem on the unsatisfiable side), but a dropped guard that other clauses needed as a *fact* can no longer be re-derived and proofs may be lost. With $\mathrm{Comp}(E)$ emitted, the dropped typing information is recoverable from the covering atom in one resolution step, and refutability is exactly preserved.

6.4 Simultaneous omission

In practice one wants to delete many guards of a premise at once, and covers may be provided by *retained guards* (the *app_nil_r* example), so licensing must be checked against the final formula. Deleting guards one at a time is already justified — each step of Definition 6.7 is licensed *in the formula it is applied to* — but the natural algorithm computes a set of deletable guards by a fixpoint and deletes them simultaneously. The next proposition shows the two views agree for antecedent guards. Call an occurrence of a subformula $\forall x.\ \mathsf{T}(x,u) \to \phi$ a *universal guard occurrence* with body ϕ.

Lemma 6.11 (Expansion)**.** *Let χ' be obtained from χ by replacing, at finitely many positions, subformulas ρ by $\mathsf{T}(s,w) \to \rho$ or by $\forall y.\ \mathsf{T}(y,w) \to \rho'$ where χ had $\forall y.\rho'$ (insertion of guard antecedents), or subformulas ρ by $\mathsf{T}(s,w) \wedge \rho$ (insertion of guard conjuncts). Then:*

(i) $\mathrm{jt}_{x,u}(\chi)$ *implies* $\mathrm{jt}_{x,u}(\chi')$*, provided only antecedent insertions are used;*

(ii) $\mathrm{jf}_{x,u}(\chi)$ *implies* $\mathrm{jf}_{x,u}(\chi')$*, provided only conjunct insertions are used.*

Proof. (i) Induction on the jt-derivation for χ. Every rule of Definition 6.4 either ignores the subformula in which the insertion occurred (the cover rule, one-sided $\vee/\to$ rules) or descends into it; in the latter case apply the induction hypothesis. If the insertion happens *at* the conclusion of the derivation — the whole judged formula ρ becomes $\mathsf{T}(s,w) \to \rho$ — then $\mathrm{jt}(\mathsf{T}(s,w) \to \rho)$ follows from $\mathrm{jt}(\rho)$ by the weakening rule $\mathrm{jt}(\phi_1 \to \phi_2) \Leftarrow \mathrm{jt}(\phi_2)$. Insertions under a quantifier that the derivation traverses are handled by the induction hypothesis at the quantifier rule's premise. (ii) Dual induction: an inserted conjunct at the judged position gives $\mathrm{jf}(\mathsf{T}(s,w) \wedge \rho)$ from $\mathrm{jf}(\rho)$ by the one-sided $\wedge$ rule. □

Proposition 6.12 (Simultaneous omission of antecedent guards)**.** *Let χ be a formula and S a set of universal guard occurrences in χ, pairwise non-overlapping in the sense that no guard atom of one occurrence lies inside another's guard atom (bodies may nest freely). Let χ_S be the result of deleting all guards in S simultaneously. If for every $g = (x_g, u_g) \in S$ the judgment $\mathrm{jt}_{x_g,u_g}(\phi_g)$ holds, where ϕ_g is the body of g as it appears in χ_S, then $\mathrm{Comp}(E) \models \chi \leftrightarrow \chi_S$; i.e. the simultaneous deletion is a valid composite of licensed steps.*

Proof. Order $S = \{g_1, \ldots, g_n\}$ arbitrarily and delete one guard per step; let χ_i denote the formula after deleting $g_1, \ldots, g_i$, so $\chi_0 = \chi$ and $\chi_n = \chi_S$. When deleting g_i from χ_{i-1}, its body ϕ' there differs from ϕ_{g_i} (its body in χ_S) exactly by the presence of the not-yet-deleted guards $g_{i+1}, \ldots, g_n$ that lie inside it — i.e. ϕ' is obtained from ϕ_{g_i} by inserting universal guard antecedents. By assumption $\mathrm{jt}_{x_{g_i},u_{g_i}}(\phi_{g_i})$, so by Lemma 6.11(i) $\mathrm{jt}_{x_{g_i},u_{g_i}}(\phi')$, and the step is licensed. Each step preserves $\mathrm{Comp}(E)$-equivalence by Theorem 6.6 and Lemma 3.4; compose. □

Remark 6.13 (Fixpoint computation, and mixing with existential guards). Proposition 6.12 is exactly what justifies the practical algorithm: start from the set of all universal guard occurrences of a premise, and iteratively remove from the candidate set any guard whose body (with all current candidates deleted) fails jt — covers being hypothesis atoms and *retained* guards. The iteration is monotone (shrinking the deleted set only adds antecedents, which by Lemma 6.11(i) preserves jt), so it reaches a greatest fixpoint, which satisfies the hypothesis of the proposition. In the *app_nil_r* example the fixpoint retains the guard on l (its body is an equation) and deletes the guard on α, covered by the retained $\mathsf{T}(l, \widehat{list}(\alpha))$. For *existential* guard conjuncts, whose licensing judgment is jf, Lemma 6.11(ii) covers simultaneous deletion of several conjuncts, but a mixed set of antecedent and conjunct deletions with nested scopes must be checked stepwise (delete innermost guards first): jf is not in general preserved by antecedent insertion, since $\mathrm{jf}(\mathsf{T}(y, w) \to \rho)$ would require the inserted guard to be junk-*true*. Stepwise licensing is always sound by Theorem 6.8.

6.5 Worked examples

Example 6.14. $[\![\forall x{:}nat.\ Even\, x \to Even\,(S(S\, x))]\!] = \forall x.\ \mathsf{T}(x, \widehat{nat}) \to (\hat{Even}(x) \to \hat{Even}(\hat{S}(\hat{S}(x))))$. The body is $A \to \psi$ with $A = \hat{Even}(x)$, and $\mathsf{T}(x, \widehat{nat}) \in \mathrm{tc}(A)$ since $\tau_1^{Even} = nat$. The cover rule gives jt; by Theorem 6.6(i) the guard may be dropped, at any polarity.

Example 6.15. In the translated inversion principle for *le* (displayed in Section 2), the guards on n and m have bodies of the form $\hat{le}(n, m) \to \cdots$ (after dropping the inner guard first, or by Lemma 6.11(i) directly), and

$\hat{le}(n,m)$ covers both $(n,\widehat{nat})$ and $(m,\widehat{nat})$: drop both. The naked disjunct $m=n$ is never inspected — the cover rule stops at the covering hypothesis. The existential conjunct $\mathsf{T}(k,\widehat{nat})$ has body $\hat{le}(n,k)\wedge m=\hat{S}(k)$, which is jf for $(k,\widehat{nat})$ by the one-sided $\wedge$ rule applied to the covering atom $\hat{le}(n,k)$; by Theorem 6.6(ii) it, too, may be dropped — and by Corollary 6.9 nothing is lost while $\mathrm{Comp}(E)$ is present.

Example 6.16. In $[\![app\_nil\_r]\!]$ (Section 2) the guard on l is *not* deletable: its body is the equation $\hat{app}(\alpha,l,\hat{nil}(\alpha))=l$, whose tc() is empty, and no rule of Definition 6.4 applies. (This is not a weakness of the criterion: the unguarded sentence is false in $\mathfrak{M}(\mathfrak{N})$ — instantiate l with any junk element; the left-hand side is then a junk tree, which differs from l itself — so no sound criterion can delete it.) The guard on α is deletable by Proposition 6.12 with S containing only it: its body in χ_S still contains the retained guard $\mathsf{T}(l,\widehat{list}(\alpha))$, an atom covering (α,Type) by the second clause of Definition 6.1.

Example 6.17. Typing axioms themselves shrink: in the axiom (T1) for *cons*, the guard $\mathsf{T}(\alpha,\mathsf{Type})$ is covered by the antecedent $\mathsf{T}(l,\widehat{list}(\alpha))$ (or by $\mathsf{T}(x,\alpha)$? — no: α there is a *variable* type side, $\mathrm{tc}(\mathsf{T}(x,\alpha))=\emptyset$; only the rigid $\widehat{list}(\alpha)$ occurrence provides the cover). For *nil*'s typing axiom $\forall\alpha.\ \mathsf{T}(\alpha,\mathsf{Type})\to\mathsf{T}(\hat{nil}(\alpha),\widehat{list}(\alpha))$ the guard must stay: the body is a T-atom, which no rule makes jt. Indeed the unguarded version is false in $\mathfrak{M}(\mathfrak{N})$: for junk α the type side is a junk tree and the T-atom fails.

7 Type-argument omission

We now treat the second optimization: erasing type arguments of function and constructor symbols. The criterion is a condition on declared types; soundness is proved by constructing an intended model of the *erased* signature in which the omitted arguments are recovered — uniquely — from the tags of the term arguments.

7.1 Criterion and erased translation

Definition 7.1 (Erasable positions)**.** Let $h\in\mathcal{C}\cup\mathcal{F}$ with declared type $\forall\alpha_1,\dots,\alpha_{k_h}.\ \tau_1^h\times\cdots\times\tau_{m_h}^h\to\tau_0^h$. A type-argument position $j\le k_h$ is *erasable* if $\alpha_j\in\mathrm{ftv}(\tau_i^h)$ for some *term* position $1\le i\le m_h$. An *erasure policy* er assigns to every $h\in\mathcal{C}\cup\mathcal{F}$ a set $\mathrm{er}(h)$ of erasable positions of h (any subset; the identity policy $\mathrm{er}\equiv\emptyset$ recovers Sections 5 and 6). Predicate symbols and type formers are never erased.

Thus for $cons:\forall\alpha.\ \alpha\times list(\alpha)\to list(\alpha)$ position 1 is erasable (α occurs in both argument types); for $nil:\forall\alpha.\ ()\to list(\alpha)$ it is not ($m_{nil}=0$);

for a phantom constant $g : \forall\alpha.\ nat \to nat$ it is not (α occurs in no argument type, only vacuously). Occurrences in the *result* type τ_0^h do not count; Proposition 8.3 shows they must not.

Remark 7.2 (Rigidity). In PFI, types are first-order terms over $\mathcal{I}$, so every occurrence of α_j in τ_i^h is *rigid*: it sits under inductive type formers only, and Lemma 7.3 below exploits exactly this. In richer calculi the criterion must additionally require rigidity of the occurrence — not under a defined type-level function, a binder, or a match — since e.g. a type function F with $F(nat) = F(bool)$ destroys the injectivity that inference relies on.

Lemma 7.3 (Injectivity of type substitution)**.** *Let $\sigma \in \mathrm{Ty}(\{\bar\alpha\})$ with $\alpha_j \in \mathrm{ftv}(\sigma)$, and let $\bar\tau, \bar\tau' \in \mathrm{GTy}^k$. If $\sigma[\bar\tau/\bar\alpha] = \sigma[\bar\tau'/\bar\alpha]$ then $\tau_j = \tau'_j$.*

Proof. Induction on σ. If $\sigma = \alpha_j$ the claim is the hypothesis. σ cannot be another variable (then $\alpha_j \notin \mathrm{ftv}(\sigma)$). If $\sigma = I(\bar\sigma)$ then $\alpha_j \in \mathrm{ftv}(\sigma_l)$ for some l, and equality of the two instances gives, since GTy is a free term algebra, componentwise equality $\sigma_l[\bar\tau/\bar\alpha] = \sigma_l[\bar\tau'/\bar\alpha]$; apply the induction hypothesis. □

Lemma 7.4 (Uniqueness of inferred type arguments)**.** *Let $h \in \mathcal{C} \cup \mathcal{F}$ and let $\bar\tau, \bar\tau' \in \mathrm{GTy}^{k_h}$ satisfy $\tau_j = \tau'_j$ for all $j \notin \mathrm{er}(h)$ and $\tau_i^h[\bar\tau/\bar\alpha] = \tau_i^h[\bar\tau'/\bar\alpha]$ for all $1 \le i \le m_h$. Then $\bar\tau = \bar\tau'$.*

Proof. For $j \in \mathrm{er}(h)$, by Definition 7.1 pick i with $\alpha_j \in \mathrm{ftv}(\tau_i^h)$ and apply Lemma 7.3 to $\sigma = \tau_i^h$. For $j \notin \mathrm{er}(h)$ the components agree by assumption. □

This is the fragment counterpart of the "inferable type argument" uniqueness lemma of Blanchette et al. [BBPS16]; in our setting it is a two-line consequence of the freeness of the type algebra, which is precisely what Remark 7.2 preserves.

Definition 7.5 (Erased signature and translation)**.** The signature Σ_E^ε is Σ_E with each $\hat h$ ($h \in \mathcal{C} \cup \mathcal{F}$) replaced by $\hat h_\varepsilon$ of arity $(k_h - |\mathrm{er}(h)|) + m_h$; predicate symbols, T, Type and the $\hat I$ are unchanged. The erasure map ε on Σ_E-terms and formulas is the homomorphic extension of

$$\varepsilon\big(\hat h(s_1, \dots, s_{k_h}, \bar t)\big) := \hat h_\varepsilon\big(\varepsilon(\bar s) \restriction \notin \mathrm{er}(h),\ \varepsilon(\bar t)\big),$$

where $\cdot \restriction \notin \mathrm{er}(h)$ keeps the components at non-erased positions in order. The *erased problem* for (E, ϕ_0) is

$$\Theta_\varepsilon(E) \cup \mathrm{Comp}(E) \cup \{\neg\, \varepsilon([\![\phi_0]\!])\}, \qquad \Theta_\varepsilon(E) := \varepsilon(\mathrm{TA}) \cup \mathrm{ID}_\varepsilon \cup \varepsilon\big([\![\mathrm{Prem}(E)]\!] \cup [\![\mathrm{Struct}(E)_\exists]\!]\big),$$

where ID_ε contains, for each constructor c, the injectivity axiom

$$\forall \dots\ \hat c_\varepsilon(\bar\alpha \restriction \notin \mathrm{er}(c), \bar x) = \hat c_\varepsilon(\bar\alpha' \restriction \notin \mathrm{er}(c), \bar x') \to \textstyle\bigwedge_{j \notin \mathrm{er}(c)} \alpha_j = \alpha'_j \wedge \bigwedge_i x_i = x'_i$$

— the equality conjuncts at *erased* positions are dropped — and the discrimination axioms with erased argument lists. Note $\varepsilon(\mathrm{Comp}(E)) = \mathrm{Comp}(E)$: completion axioms mention only predicate atoms, T, and translated types, none of which are touched by ε.

The special-casing of ID_ε is forced: the blind erasure of the injectivity axiom would retain a conjunct $\alpha_j = \alpha'_j$ whose variables no longer occur in the antecedent, yielding a sentence that entails $\forall \alpha\, \alpha'.\ \alpha = \alpha'$ and is false in every model with more than one element. Erasure removes exactly the parameter-injectivity claims that the omitted arguments used to carry.

7.2 The erased intended models

Fix $\mathfrak{N} \models \mathbb{T}(E)$. The structure $\mathfrak{M}_\varepsilon(\mathfrak{N})$ is built by the same recipe as $\mathfrak{M}(\mathfrak{N})$ (Definitions 5.4 and 5.5), over the signature Σ_E^ε: the domain is $D_\varepsilon := D_0 \uplus J_\varepsilon$ with D_0 unchanged and J_ε the free junk universe over the Σ_E^ε-symbols, and Type, the $\hat{I}$, T and the $\hat{p}$ are interpreted verbatim as in Definition 5.5. Only the erased function symbols need a new clause:

Definition 7.6 (Interpretation of erased symbols)**.** For $h \in \mathcal{C} \cup \mathcal{F}$ let $n := k_h - |\mathrm{er}(h)|$ and let $\bar{e} \in D_\varepsilon^n$, $\bar{d} \in D_\varepsilon^{m_h}$. Say that $\bar{\tau} \in \mathrm{GTy}^{k_h}$ *matches* $(\bar{e}, \bar{d})$ if $\bar{\tau} \restriction \notin \mathrm{er}(h) = \bar{e}$ (in particular each $e_l \in \mathrm{GTy}$) and $d_i = (\tau_i^h[\bar{\tau}/\bar{\alpha}], a_i)$ with $a_i \in \mathfrak{N}_{\tau_i^h[\bar{\tau}/\bar{\alpha}]}$ for all $i \le m_h$. By Lemma 7.4 at most one $\bar{\tau}$ matches. Define

$$\hat{h}_\varepsilon^{\mathfrak{M}_\varepsilon(\mathfrak{N})}(\bar{e}, \bar{d}) := \begin{cases} \big(\tau_0^h[\bar{\tau}/\bar{\alpha}],\ h_{\bar{\tau}}^{\mathfrak{N}}(\bar{a})\big) & \text{if some (unique) } \bar{\tau} \text{ matches } (\bar{e}, \bar{d}), \\ (\hat{h}_\varepsilon, \bar{e}\,\bar{d}) \in J_\varepsilon & \text{otherwise.} \end{cases}$$

Lemma 7.7 (Erased denotation and relativization)**.** *Let ϕ, t, σ be well formed in $(V;\Gamma)$, let $\theta : V \to \mathrm{GTy}$ and let v be a proper valuation, with $w_{\theta,v}$ as before (now valued in D_ε). Then:*

(i) $[\![\sigma]\!]^{\mathfrak{M}_\varepsilon(\mathfrak{N}),w_{\theta,v}} = \sigma\theta$, and $\varepsilon([\![t]\!])^{\mathfrak{M}_\varepsilon(\mathfrak{N}),w_{\theta,v}} = (\tau\theta, t\theta^{\mathfrak{N},v})$ where τ is the type of t;

(ii) for type-quantifier-free ϕ: $\mathfrak{M}_\varepsilon(\mathfrak{N}), w_{\theta,v} \models \varepsilon([\![\phi]\!])$ iff $\mathfrak{N}, v \models \phi\theta$; and for every sentence ϕ_0: $\mathfrak{M}_\varepsilon(\mathfrak{N}) \models \varepsilon([\![\phi_0]\!])$ iff $\mathfrak{N} \models_{\mathrm{g}} \phi_0$.

Proof. (i) Types are unchanged by ε and their denotation clause is verbatim that of $\mathfrak{M}(\mathfrak{N})$; the first claim is Lemma 5.6's type part. For terms, induction: the variable case is the definition of $w_{\theta,v}$. For $t = h\langle\bar{\sigma}\rangle(\bar{t})$, the retained type arguments denote $\bar{\sigma}\theta \restriction \notin \mathrm{er}(h)$ and, by the induction hypothesis, the term arguments denote $(\tau_i^h[\bar{\sigma}\theta/\bar{\alpha}], t_i\theta^{\mathfrak{N},v})$. Hence $\bar{\tau} := \bar{\sigma}\theta$ matches, is the unique match, and the first clause of Definition 7.6 returns $(\tau_0^h[\bar{\sigma}\theta/\bar{\alpha}], (h\langle\bar{\sigma}\rangle(\bar{t}))\theta^{\mathfrak{N},v})$ as required.

(ii) The induction of Lemma 5.7 goes through verbatim: atoms and equations use (i) (predicate atoms are untouched by ε except inside their

argument terms), the guard analysis in the quantifier cases uses only T, Type and type denotations, which are unchanged. □

Theorem 7.8 (Model correctness, erased)**.** $\mathfrak{M}_\varepsilon(\mathfrak{N}) \models \Theta_\varepsilon(E) \cup \mathrm{Comp}(E)$.

Proof. $\varepsilon(\mathrm{TA})$. Consider the erased typing axiom of h; its quantifier prefix and guards are unchanged (all k_h type variables are still quantified and guarded — only the term $\hat{h}(\bar{\alpha}, \bar{x})$ shrank). Assume the guards under a valuation w: then $w(\bar{\alpha}) = \bar{\tau} \in \mathrm{GTy}^{k_h}$ and $w(x_i) = (\tau_i^h[\bar{\tau}/\bar{\alpha}], a_i)$ as in the proof of Theorem 5.8(T1). Then $\bar{\tau}$ matches $(\bar{\tau} \restriction \notin \mathrm{er}(h), w(\bar{x}))$, so $\hat{h}_\varepsilon^{\mathfrak{M}_\varepsilon(\mathfrak{N})}$ returns $(\tau_0^h[\bar{\tau}/\bar{\alpha}], h_{\bar{\tau}}^{\mathfrak{N}}(\bar{a}))$ and the conclusion T-atom holds. The $\hat{I}$ axioms are unchanged.

ID_ε. As in Theorem 5.8(T2), with three cases on the two values. If both are proper, their tags are $I(\bar{\tau})$ and $I(\bar{\tau}')$ for the (unique) matching full instantiations; equality of tags gives $\bar{\tau} = \bar{\tau}'$, whence the retained conjuncts $\alpha_j = \alpha'_j$ ($j \notin \mathrm{er}(c)$, components of $\bar{\tau}, \bar{\tau}'$) and, via $\mathfrak{N} \models_{\mathrm{g}}$ (S1), $\bar{x} = \bar{x}'$. If both are junk, freeness of J_ε gives componentwise equality of the (erased) argument tuples, which contains every retained conjunct. Mixed cases falsify the antecedent. Discrimination: distinct heads or (S2) as before.

ε of translated sentences. Each is $\varepsilon([\![\phi]\!])$ with $\mathfrak{N} \models_{\mathrm{g}} \phi$; apply Lemma 7.7(ii).

$\mathrm{Comp}(E)$. The interpretations of $\hat{p}$, T, Type, $\hat{I}$ in $\mathfrak{M}_\varepsilon(\mathfrak{N})$ are those of Definition 5.5, so the proofs of (C1) and (C2) in Theorem 5.8 apply unchanged. □

7.3 Soundness and composition

Theorem 7.9 (Soundness of type-argument omission)**.** *For every erasure policy* er*: if* $\Theta_\varepsilon(E) \cup \mathrm{Comp}(E) \cup \{\neg\, \varepsilon([\![\phi_0]\!])\}$ *is unsatisfiable, then* $\mathbb{T}(E) \models_{\mathrm{g}} \phi_0$.

Proof. Contrapositive, as in Theorem 5.9: from $\mathfrak{N} \models \mathbb{T}(E)$ with $\mathfrak{N} \not\models_{\mathrm{g}} \phi_0$, Theorem 7.8 and Lemma 7.7(ii) exhibit $\mathfrak{M}_\varepsilon(\mathfrak{N})$ as a model of the erased problem. □

Theorem 7.10 (Composition)**.** *All results of Section 6 hold verbatim for the erased problem: the typing consequences* $\mathrm{tc}(\cdot)$*, covers, and the judgments* jt/jf *are read over* Σ_E^ε *with the* same *defining clauses (predicate atoms retain their full argument lists, and* T*-atoms their translated types, so Definition 6.1 is unchanged), and any optimized problem obtained from* $\Theta_\varepsilon(E) \cup \mathrm{Comp}(E) \cup \{\neg\, \varepsilon([\![\phi_0]\!])\}$ *by the steps of Definition 6.7 satisfies: unsatisfiability implies* $\mathbb{T}(E) \models_{\mathrm{g}} \phi_0$*. Moreover Corollary 6.9 holds for the erased problem.*

Proof. Lemmas 3.4, 6.2, 6.5 and 6.11, Theorem 6.6, and Proposition 6.12 are statements about arbitrary structures satisfying $\mathrm{Comp}(E)$ and never mention the function symbols $\hat{h}$; they hold over Σ_E^ε with identical proofs. The soundness argument of Theorem 6.8 then runs with $\mathfrak{M}_\varepsilon(\mathfrak{N})$ in place of

$\mathfrak{M}(\mathfrak{N})$, using $\mathfrak{M}_\varepsilon(\mathfrak{N}) \models \mathrm{Comp}(E)$ (Theorem 7.8) and Lemma 7.7(ii) for the goal. Corollary 6.9's proof is signature-independent. □

Remark 7.11 (Why predicates are not erased). The composition theorem depends on predicates keeping their type arguments: the completion axiom $\forall \bar{\alpha}\, \bar{x}.\ \hat{p}(\bar{\alpha}, \bar{x}) \to \mathsf{T}(x_i, [\![\tau_i^p]\!])$ mentions $\bar{\alpha}$ on both sides, and erasing $\hat{p}$'s type arguments would leave a completion "axiom" quantifying a type variable occurring only in the conclusion — which is false in the natural erased interpretation of $\hat{p}$ (an atom $\hat{p}_\varepsilon(\bar{x})$ can only assert *existence* of a matching instance, not pin the specific $\bar{\alpha}$). Covers, and with them the entire guard-omission apparatus, would silently lose their justification; Proposition 8.4 shows the resulting system is actually unsound, not merely unproven. Since atom sizes are not where ATP term-indexing pain concentrates (term bloat lives in constructor and function spines inside equations), restricting erasure to $\mathcal{C} \cup \mathcal{F}$ costs little.

Remark 7.12 (Choosing the policy). Theorems 7.9 and 7.10 hold for *every* er within the erasable positions, so an implementation is free to erase fewer arguments than allowed — e.g. to intersect the erasable positions with the positions the proof assistant itself marks as implicit, which matches user-visible syntax and keeps translated problems readable. What it must not do is treat implicitness as the criterion: elaboration also infers *result-only* arguments (those of *nil*) from the expected type, an information channel with no counterpart at a first-order term occurrence (Proposition 8.3).

8 Tightness of the criteria

Each clause of the criteria excludes something, and each exclusion is necessary: relaxing it admits a consistent environment whose transformed translation refutes a goal not entailed by the source theory. We record one counterexample per exclusion. In all four propositions "unsound" means: the transformed problem $\Theta' \cup \mathrm{Comp}(E) \cup \{\neg G'\}$ is unsatisfiable although $\mathbb{T}(E) \not\models_{\mathrm{g}} \phi_0$ — the negation of the guarantee provided by Theorems 6.8, 7.9 and 7.10.

Proposition 8.1 (Equations do not cover: uncovered guards cannot be dropped)**.** *Let E_1 have inductive types unit (constructor tt) and bool (constructors true, false), no functions or predicates, and the single premise $L = \forall x{:}\mathit{unit}.\ x \approx_{\mathit{unit}} \mathit{tt}$. Let $\phi_0 = \mathit{true} \approx_{\mathit{bool}} \mathit{false}$. Then E_1 is consistent, $\mathbb{T}(E_1) \not\models_{\mathrm{g}} \phi_0$, and replacing $[\![L]\!] = \forall x.\ \mathsf{T}(x, \widehat{\mathit{unit}}) \to x = \hat{\mathit{tt}}$ by the unguarded $\forall x.\ x = \hat{\mathit{tt}}$ makes the problem unsatisfiable.*

Proof. Consistency and non-entailment: the ground structure $\mathfrak{N}$ with $\mathfrak{N}_{\mathit{unit}} = \{\bullet\}$, $\mathfrak{N}_{\mathit{bool}} = \{0, 1\}$ and the evident constructor interpretations satisfies $\mathbb{T}(E_1)$ (injectivity, distinctness and exhaustiveness hold in the standard interpretations; L holds) and falsifies ϕ_0. Unsatisfiability of the transformed

problem: from $\forall x.\ x = \hat{tt}$ instantiate x with $\hat{true}$ and $\hat{false}$ (both closed terms) to get $\hat{true} = \hat{tt} = \hat{false}$; the discrimination axiom $\hat{true} \neq \hat{false}$ of ID yields $\bot$, so the problem is unsatisfiable regardless of the goal. The criterion of Definition 6.4 rejects the step: the body $x = \hat{tt}$ is an equation, $\mathrm{tc}(x = \hat{tt}) = \emptyset$ (Definition 6.1), and no rule applies. Hence the clause $\mathrm{tc}(\text{equation}) = \emptyset$ cannot be relaxed to let a variable's occurrence in an equation count as a cover. □

This is the fragment form of the classic finite-exhaustion unsoundness of unguarded translations [MP08, CK18]: a one-element type exports its extensionality to the whole untyped domain.

Proposition 8.2 (Existential guards need jf, not jt)**.** *Let E_2 have the single inductive type void with no constructors and no premises, and let $\phi_0 = \exists x{:}void.\ \top$. Then E_2 is consistent, $\mathbb{T}(E_2) \not\models_{\mathrm{g}} \phi_0$, and rewriting the goal translation $[\![\phi_0]\!] = \exists x.\ \mathsf{T}(x, \widehat{void}) \wedge \top$ to $\exists x.\ \top$ — which the* universal *judgment would license, since $\mathrm{jt}_{x,\widehat{void}}(\top)$ holds — makes the problem unsatisfiable.*

Proof. The ground structure with $\mathfrak{N}_{void} = \emptyset$ satisfies $\mathbb{T}(E_2)$ — exhaustiveness for *void* is $\forall z{:}void.\ \bot$, vacuously true — and falsifies ϕ_0. The transformed problem contains $\neg\exists x.\ \top$, which is unsatisfiable on its own (domains are nonempty). The criterion rejects the step: Theorem 6.6(ii) demands $\mathrm{jf}_{x,\widehat{void}}(\top)$, and no rule derives $\mathrm{jf}(\top)$. Hence the $\forall/\exists$ asymmetry of Theorem 6.6 is essential: junk-truth of the body makes a guarded *universal* redundant but makes a guarded *existential* strictly stronger than its unguarded form. □

Proposition 8.3 (Result-only type arguments are not erasable)**.** *Let E_3 have inductive types nat (constructors zero, succ), unit, bool (as above), a function constant card : $\forall\alpha.\ () \to nat$ (no term arguments), and premises*

$$L_1 = card\langle unit\rangle() \approx_{nat} succ(zero), \qquad L_2 = card\langle bool\rangle() \approx_{nat} zero.$$

Let $\phi_0 = succ(zero) \approx_{nat} zero$. Then E_3 is consistent, $\mathbb{T}(E_3) \not\models_{\mathrm{g}} \phi_0$, and erasing card's type argument — forbidden by Definition 7.1, as position 1 occurs in no term argument type — makes the problem unsatisfiable.

Proof. Consistency: the standard structure with $card_{unit} = 1$, $card_{bool} = 0$ (and standard *nat*) satisfies both premises and the structural axioms and falsifies ϕ_0. After the illegal erasure, both premises speak about the single constant $\hat{card}_\varepsilon$: $\hat{card}_\varepsilon = \hat{succ}(\hat{zero})$ and $\hat{card}_\varepsilon = \hat{zero}$, whence $\hat{succ}(\hat{zero}) = \hat{zero} = [\![\phi_0]\!]$; the problem containing $\neg[\![\phi_0]\!]$ is unsatisfiable. (Where the soundness proof would break: Lemma 7.4 is inapplicable — no term position witnesses α — so Definition 7.6 has no well-defined proper clause for $\hat{card}_\varepsilon$, and any total choice falsifies L_1 or L_2 in the erased model.) This

is the formal core of the *card* UNIV unsoundness of [MP08] and of the *nil* example: erasing *nil*'s argument likewise conflates instances, with the collapse surfacing through premises such as $rev\langle\alpha\rangle(nil\langle\alpha\rangle) \approx nil\langle\alpha\rangle$ at two instances. □

Proposition 8.4 (Erasing predicate type arguments breaks covers)**.** *Let E_4 have inductive types nat, bool (as above) and list (constructors $nil : \forall\alpha.\ () \to list(\alpha)$, $cons : \forall\alpha.\ \alpha \times list(\alpha) \to list(\alpha)$), a predicate $q : \forall\alpha.\ \alpha \to \mathsf{Prop}$, and premises*

$$P_1 = \forall l{:}list(nat).\ q\langle list(nat)\rangle(l) \to l \approx_{list(nat)} nil\langle nat\rangle(),$$
$$P_2 = q\langle list(bool)\rangle\big(cons\langle bool\rangle(true, nil\langle bool\rangle())\big).$$

Let $\phi_0 = true \approx_{bool} false$. Then E_4 is consistent, $\mathbb{T}(E_4) \not\models_{\mathrm{g}} \phi_0$, and the combination of (a) erasing q's type argument — forbidden, since Definition 7.1 restricts er *to $\mathcal{C} \cup \mathcal{F}$ — and (b) dropping the guard of P_1's translation on the strength of the "cover" $\hat{q}_\varepsilon(l)$ makes the problem unsatisfiable.*

Proof. Consistency: interpret $q_{list(nat)}$ as $\{nil\}$, $q_{list(bool)}$ as $\{[true]\}$, all other instances as empty, over the standard list structures; P_1, P_2 and the structural axioms hold, and ϕ_0 fails. After the two illegal steps the problem contains

$$\forall l.\ \hat{q}_\varepsilon(l) \to l = \hat{nil}(\widehat{nat}) \qquad \text{and} \qquad \hat{q}_\varepsilon\big(\hat{cons}(\widehat{bool}, \hat{true}, \hat{nil}(\widehat{bool}))\big).$$

Instantiating the former with the latter's argument yields $\hat{cons}(\widehat{bool}, \hat{true}, \hat{nil}(\widehat{bool})) = \hat{nil}(\widehat{nat})$, contradicting the unguarded discrimination axiom $\forall\alpha\, x\, l\, \alpha'.\ \hat{cons}(\alpha, x, l) \neq \hat{nil}(\alpha')$ of ID; the problem is unsatisfiable regardless of the goal. Where the soundness proof breaks: after erasing q, no completion axiom of the form $\forall\alpha\, x.\ \hat{q}_\varepsilon(x) \to \mathsf{T}(x, \alpha\text{-dependent type})$ is even expressible — the natural interpretation of $\hat{q}_\varepsilon(d)$ is "d satisfies *some* instance of q", which forces no particular type — so $\hat{q}_\varepsilon(l)$ is not strict and covers nothing. With guards intact (only step (a)), or with q unerased (only step (b), which is then licensed and sound by Theorem 7.10), no contradiction arises. □

Remark 8.5. The four propositions delimit the design space sharply: covers must be atoms whose predicate is *false-extended* in the intended model (equations and non-rigid T-atoms are not), existential guards are facts rather than obligations, type-argument recovery needs a witness among the *retained term arguments*, and the two optimizations compose only because they partition the signature (predicates feed guard omission; functions and constructors feed erasure). Every known practical unsoundness pattern of unguarded or type-erased hammer translations that falls inside our fragment is an instance of one of the four [MP08, BBPS16, CK18].

9 Related work

Type encodings for polymorphic logics. The systematic study of sound type erasure begins with Meng and Paulson [MP08], who documented both the performance cost of full type annotations and the concrete unsoundness of dropping them (Propositions 8.1 and 8.3 are fragment forms of their examples), and proposed partial encodings validated empirically. Couchot and Lescuyer [CL07] and Bobot and Paskevich [BP11] gave provably sound guard- and tag-based encodings of polymorphic FOL into untyped and many-sorted FOL. Claessen, Lilliestr\"om and Smallbone [CLS11] introduced monotonicity-based erasure for many-sorted logic; their "calculus 2", in which an arbitrary predicate may serve as the guard of a sort provided it can consistently be extended to be false outside it, is the direct formal ancestor of our covers. Our Definition 5.5 builds the false extension into the intended model once and for all — an ill-typed atom is false because an ill-typed application is not a provable proposition — so no per-problem satisfiability check is needed; the price is that atoms which are not false-extended in $\mathfrak{M}(\mathfrak{N})$ (equations, non-rigid T-atoms) can never serve as covers, which Section 8 shows is essential anyway. Blanchette, Böhme, Popescu and Smallbone [BBPS16] developed the most comprehensive framework: their *covers* and *inferable type arguments* correspond to our Definition 7.1 and Lemma 7.4, their *featherweight guards* $\mathsf{g}??$ — guards asserted only for naked variables — are the satisfiability-preserving relative of our per-formula equivalence, and their monotonicity calculi point to a third, complementary omission principle that our development does not cover (see below). Their encodings, implemented in Sledgehammer and, in guard form, in Why3 [FP13], are global problem transformations proved sound by model constructions similar in spirit to Section 5.2; the difference in our setting is dependency — the forced type $[\![\tau_i^p]\!][\bar{s}/\bar{\alpha}]$ of an argument position may mention the other arguments — and the conclusion is correspondingly finer-grained: a per-occurrence logical equivalence relative to $\mathrm{Comp}(E)$ rather than the equisatisfiability of a whole-problem encoding.

Hammers and CIC. For the hammer context see the survey [BKPU16] and the systems for HOL [PB12, KU14] and Mizar [KU15]. CoqHammer [CK18] defined the translation whose optimization we study; its practically tuned variant already omits selected guards and justifies the choices empirically, by counterexample-driven testing and reconstruction rates. The theoretical companion [Cza18] proves such a translation of a class of pure type systems sound *proof-theoretically* — any FOL proof from the translated axioms maps to a source derivation — but for the guarded variant only; omission of redundant guards is stated there as a conjecture, and the proof hinges on an invariant that guards are the sole channel through which typability information flows into a FOL proof. Our results give the

classical, model-theoretic counterpart for a fragment with inductive types, and suggest a route to the proof-theoretic version: Corollary 6.9 shows that a proof from guard-omitted axioms converts, purely in FOL, into a proof from the guarded axioms plus Comp(E), so it suffices to extend the invariant to completion axioms — whose source reading is exactly typing inversion: from a reconstruction of a proof of $p\,\bar{\sigma}\,\bar{t}$ one extracts typing evidence for $\bar{t}$. Carrying this out in the setting of [Cza18] (which would first need extending that framework to inductive types) remains future work. Mizar's *conditional cluster registrations*, exported to ATPs by MPTP [Urb06], are Horn sentences of exactly the completion-axiom shape ("every *cardinal* is an *ordinal*"), used at scale in the MizAR hammer [KU15]; our Comp(E) can be read as the automatically generated cluster theory of the translated environment. In the Lean ecosystem, Lean-auto [QCBA25] faces the same design choice for its monomorphizing translation. On the source side, our erasability criterion parallels Letouzey's program extraction [Let03]: erasable type arguments are those recoverable from the extracted (term-level) data, and result-only arguments are exactly the positions where extraction must insert an unchecked coercion.

Limitations and future work. The fragment PFI isolates the polymorphic first-order core of CIC, and three omissions matter in practice. (i) *Conversion*: CoqHammer emits lambda-lifted definitional equations *unguarded*, justified by a different intended semantics in which equality is convertibility of (possibly ill-typed) terms; our models interpret equality as identity on tagged values and junk trees, so unguarded definitional equations are outside the present theorems (and indeed our criterion conservatively keeps their guards unless a cover is present). Reconciling the two semantics — a quotient of the junk universe by an untyped conversion — is the natural next step. (ii) *Dependent data and higher-order features*: type formers with term indices, function types as data, and non-prenex polymorphism all break the freeness arguments in Lemma 7.3 and Definition 5.5; hammer translations handle them by lifting and collapsing, and the interaction of those devices with omission (in particular, which lifted atoms are false-extended and hence covers) is open — the known constraint being that atoms headed by lifted proposition constants must *not* count as covers, since their defining equivalences quantify unguarded. (iii) *Monotonicity*: for types whose translated theory cannot distinguish domain sizes, guards can be omitted with *no* cover at all [CLS11, BBPS16]; adapting monotonicity inference to a dependently typed source, where the finiteness of a type can depend on term parameters, is the open problem already posed in [Cza18], and our Proposition 8.1 (a one-element type) marks exactly the non-monotonic case that any such analysis must detect.

10 Conclusion

We have given syntactic criteria for the two dominant redundancies of guarded hammer translations of CIC — type guards covered by a strict hypothesis atom, and type arguments recoverable from retained term arguments — and proved them sound, and in the guard case conservative, for a polymorphic first-order fragment of CIC with inductive definitions. The proofs are self-contained model constructions; the single point of contact with CIC metatheory is the (cited) existence of a classical model of the source environment, and the single semantic insight doing the work is that the intended interpretation of a translated predicate is *false-extended*: an ill-typed application is not a provable proposition. The criteria evaluate on the emitted first-order formulas using only the declared types of the translated constants, and are therefore implementable as a post-processing pass in an existing hammer; the completion axioms $\mathrm{Comp}(E)$, themselves sound to emit, make guard omission lossless. The tightness examples delimit what any extension must respect: covers must be false-extended atoms, existential guards are facts, inference needs a retained witness, and predicates must keep the arguments that make them strict.

References

[BBPS16] Jasmin Christian Blanchette, Sascha Böhme, Andrei Popescu, and Nicholas Smallbone. Encoding monomorphic and polymorphic types. *Logical Methods in Computer Science*, 12(4:13):1–52, 2016.

[BKPU16] Jasmin Christian Blanchette, Cezary Kaliszyk, Lawrence C. Paulson, and Josef Urban. Hammering towards QED. *Journal of Formalized Reasoning*, 9(1):101–148, 2016.

[BP11] François Bobot and Andrei Paskevich. Expressing polymorphic types in a many-sorted language. In *Frontiers of Combining Systems (FroCoS 2011)*, volume 6989 of *Lecture Notes in Computer Science*, pages 87–102. Springer, 2011.

[CK18] Łukasz Czajka and Cezary Kaliszyk. Hammer for Coq: Automation for dependent type theory. *Journal of Automated Reasoning*, 61(1–4):423–453, 2018.

[CL07] Jean-François Couchot and Stéphane Lescuyer. Handling polymorphism in automated deduction. In *Automated Deduction – CADE-21*, volume 4603 of *Lecture Notes in Computer Science*, pages 263–278. Springer, 2007.

[CLS11] Koen Claessen, Ann Lillieström, and Nicholas Smallbone. Sort it out with monotonicity: Translating between many-sorted and unsorted first-order logic. In *Automated Deduction – CADE-23*, volume 6803 of *Lecture Notes in Computer Science*, pages 207–221. Springer, 2011.

[Cza18] Łukasz Czajka. A shallow embedding of pure type systems into first-order logic. In *22nd International Conference on Types for Proofs and Programs (TYPES 2016)*, volume 97 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 9:1–9:39. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018.

[FP13] Jean-Christophe Filliâtre and Andrei Paskevich. Why3 — where programs meet provers. In *Programming Languages and Systems (ESOP 2013)*, volume 7792 of *Lecture Notes in Computer Science*, pages 125–128. Springer, 2013.

[KU14] Cezary Kaliszyk and Josef Urban. Learning-assisted automated reasoning with Flyspeck. *Journal of Automated Reasoning*, 53(2):173–213, 2014.

[KU15] Cezary Kaliszyk and Josef Urban. MizAR 40 for Mizar 40. *Journal of Automated Reasoning*, 55(3):245–256, 2015.

[Let03] Pierre Letouzey. A new extraction for Coq. In *Types for Proofs and Programs (TYPES 2002)*, volume 2646 of *Lecture Notes in Computer Science*, pages 200–219. Springer, 2003.

[LW11] Gyesik Lee and Benjamin Werner. Proof-irrelevant model of CC with predicative induction and judgmental equality. *Logical Methods in Computer Science*, 7(4:5):1–25, 2011.

[MP08] Jia Meng and Lawrence C. Paulson. Translating higher-order clauses to first-order clauses. *Journal of Automated Reasoning*, 40(1):35–60, 2008.

[PB12] Lawrence C. Paulson and Jasmin Christian Blanchette. Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. In *Proceedings of the 8th International Workshop on the Implementation of Logics (IWIL 2010)*, volume 2 of *EPiC Series in Computing*, pages 1–11. EasyChair, 2012.

[QCBA25] Yicheng Qian, Joshua Clune, Clark Barrett, and Jeremy Avigad. Lean-auto: An interface between Lean 4 and automated theorem provers. arXiv:2505.14929, 2025.

[Urb06] Josef Urban. MPTP 0.2: Design, implementation, and initial experiments. *Journal of Automated Reasoning*, 37(1–2):21–43, 2006.

[Wer97] Benjamin Werner. Sets in types, types in sets. In *Theoretical Aspects of Computer Software (TACS 1997)*, volume 1281 of *Lecture Notes in Computer Science*, pages 530–546. Springer, 1997.